YOCTO PROJECT · SCARTHGAP 5.0 · BEAGLEBONE BLACK

Yocto by Construction

Twenty recipe-writing assignments. You write the code, you write the `.bb`, and a test tells you whether you got it right.

For Dipesh · build host Ubuntu 24.04 · machine `beaglebone-yocto`

Follows the 10-recipe reading ladder (makedevs → lttng-modules)

Prepared 17 October 2026

PHASE 1 Recipe foundations · A01–A04

PHASE 2 Build systems & configurability · A05–A09

PHASE 3 Integrating with the platform · A10–A12

PHASE 4 Kernel space · A13–A15

PHASE 5 Developer workflows · A16–A17

PHASE 6 Product: policy, storage, verification · A18–A20

H build-host tests: the metadata does what you think

T target tests: it works on the BeagleBone Black

B break-it tests: predict the failure before you cause it

An assignment is done when every H, T and B test passes, not when bitbake exits 0.

Contents

How this curriculum works 3

A00 · Lab setup (do this once) 5

PHASE 1 · RECIPE FOUNDATIONS

A01 · Your layer and your first recipe 7

A02 · An upstream project: bbbctl 0.1 from git 9

A03 · A shared library from scratch: libbbbhw 0.1 11

A04 · Consuming the library: bbbctl 0.2 13

PHASE 2 · BUILD SYSTEMS & CONFIGURABILITY

A05 · Autotools from scratch: bbbinfo 1.0 15

A06 · CMake: bbb-ledd 0.1 17

A07 · Native tools and build-time code generation: bbb-patc 19

A08 · Optional features: PACKAGECONFIG and bbbinfo 1.1 21

A09 · Services and system users: bbb-ledd 0.3 23

PHASE 3 · INTEGRATING WITH THE PLATFORM

A10 · Changing recipes you don't own: bbappends 25

A11 · Your image: bbb-lab-image and packagegroup-bbb-lab 27

A12 · Package feeds, versions and on-target upgrades 29

PHASE 4 · KERNEL SPACE

A13 · Kernel configuration fragments 31

A14 · An out-of-tree kernel module: bbblab.ko 33

A15 · Device tree and a platform driver: binding bbblab to hardware 35

PHASE 5 · DEVELOPER WORKFLOWS

A16 · devtool: the edit, build, deploy loop 37

A17 · The SDK: developing outside BitBake 39

PHASE 6 · PRODUCT: POLICY, STORAGE, VERIFICATION

A18 · Your own distro and machine 41

A19 · Storage layout: custom WIC, read-only rootfs, persistent /data 43

A20 · Capstone: test automation with ptest and testimage 45

AFTER A20

Mastery roadmap 47

Appendix A · Command reference 48

Appendix B · Where things live (scarthgap, beaglebone-yocto) 49

Appendix C · Concept index 50

Appendix D · Verification notes 51

How this curriculum works

Your previous assignment, the ten-recipe reading ladder (makedevs, libaio, which, acl/attr, expat, psplash, busybox, u-boot, linux-yocto, lttng-modules), trained **recognition**: you read recipes and traced them into classes. This curriculum trains **production**. In every assignment you write the source code, put it in a git repository, write the recipe from an empty file, and prove it works with tests on the build host and on the board.

Across the twenty assignments you build one small product, the **bbb-lab** system. It has a hardware library, three applications, a daemon, a build-time code generator, a kernel driver with a device-tree binding, an image, a distro, a machine, a storage layout and an automated test suite. Each assignment adds one layer of Yocto knowledge on top of the previous ones.

The rules

1. **One new concept per assignment.** Everything else is reuse. Each assignment header names the *new concept* and what it *builds on*. If you get stuck on something that is not the new concept, the gap is in an earlier assignment. Go back and fix it there.
2. **Write the `.bb` from an empty file.** Do not copy a recipe from `meta/`. You may *read* `meta-skeleton`, the classes and any recipe you like. Typing every line yourself is what makes it stick.
3. **Predict before you run.** Every *Break it* test asks you to write down the predicted failure before you trigger it. A wrong prediction is the most useful result you can get. It points at the exact mental model that needs repair.
4. **Tests define done.** An assignment is finished when every H, T and B test passes, not when `bitbake` exits 0. Many of the traps in this curriculum build successfully and are wrong.
5. **Keep a lab notebook.** For each assignment, record each prediction, what actually happened, and one sentence on why.

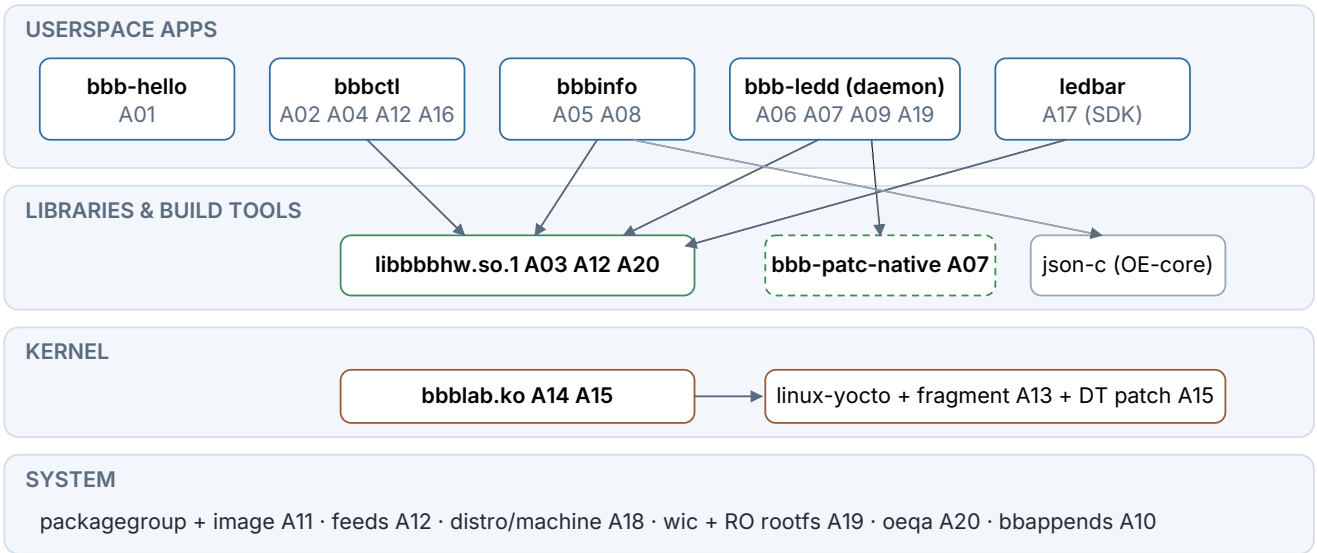
Test types and IDs

Type	Where it runs	What it proves
H: build host	your build shell, after sourcing the build environment	the metadata does what you think: variables, packages, sysroots, QA
T: target	on the BBB, over ssh or serial	the product works on real hardware
B: break it	build host, sometimes the target	you understand *why* it works, because you can predict how it fails

Every test has an ID such as `A05-H2`. Quote the ID when you ask a question about it. Where an exact string depends on your tree (kernel release, git hash, IP address), the *Expect* line says what to compare against instead of giving a literal.

> **How tests are written.** Commands use the helpers defined in A00 below: `$SRC`, `$LAYER`, `$BBB`, `$LED`, `bbb …` (run on the board) and `var RECIPE VARIABLE` (print one variable of one recipe). Where an expected value comes from your own tree, the test tells you how to get it rather than asserting a number.

The product you will build



Solid arrows: link-time or runtime dependency. Dashed: build-time only (a native tool that runs on your PC). Assignment numbers show where each component is created or changed.

Curriculum map

#	Assignment	You build	New concept	Builds on	Ladder link
A01	Your layer, first recipe	`bbb-hello` (1 C file)	layer.conf, FILESPATH, `S` / `D`, `CC`, licensing	–	makedevs
A02	An upstream project	`bbbctl` 0.1 (Makefile)	git fetcher, SRCREV, PV `+git`, `oe_runmake`, DESTDIR	A01	libaio
A03	A shared library	`libbbbhw` 0.1	SONAME, package split, sysroot staging, debian renaming	A02	libaio, acl
A04	Consuming the library	`bbbctl` 0.2	DEPENDS, recipe sysroot, pkg-config, shlibs RDEPENDS	A03	acl/attr
A05	Autotools from scratch	`bbbinfo` 1.0	autotools.bbclass, EXTRA_OECONF, `B` ≠ `S`, configure QA	A04	which, acl
A06	CMake	`bbb-ledd` 0.1	cmake.bbclass, toolchain file, EXTRA_OECMAKE	A04	expat
A07	Native tools	`bbb-patc` + `bbb-ledd` 0.2	`-native`, BBCLASSEXTEND, build-time code generation	A06	–
A08	Optional features	`bbbinfo` 1.1 (+json-c)	PACKAGECONFIG, task signatures, `bitbake-diffsigs`	A05	acl
A09	Services & system users	`bbb-ledd` 0.3	update-rc.d, systemd, useradd, CONFFILES, pseudo	A07, A08	psplash
A10	Changing recipes you don't own	base-files, busybox	bbappend, FILESPATH search order, config fragments	A01	busybox
A11	Your image	`bbb-lab-image`	image recipe, packagegroup, rootfs hooks, buildhistory, license policy	A01–A10	–
A12	Feeds and upgrades	`libbbbhw` 0.2, `bbbctl` 0.3	package-index, PR service, PKGV/PKGR, version QA	A11	–
A13	Kernel configuration	linux-yocto bbappend	config fragments, `kernel_configcheck`	A10	linux-yocto
A14	Out-of-tree module	`bbblab.ko`	module.bbclass, autoload, probeconf	A13	lttng-modules
A15	Device tree + driver	DT patch, `bbblab` 0.2	kernel patches, dtb deploy, modalias autoload	A14	linux-yocto, u-boot
A16	devtool	`bbbctl` change	workspace, externalsrc, deploy-target, finish	A02–A15	–
A17	The SDK	`ledbar` app	populate_sdk, environment-setup, `devtool add`	A11, A16	–
A18	Distro and machine	`bbblab` distro, `bbb-lab` machine	policy vs hardware, MACHINEOVERRIDES, systemd	A09, A11	u-boot
A19	Storage layout	custom `.wks`, RO rootfs	wic, read-only-rootfs, persistent `/data`	A18	–
A20	Capstone: test automation	ptest + oeqa suite	ptest, testimage on hardware, image variants	all	–

A00 · Lab setup (do this once)

> **Why this exists (Chesterton's Fence).** Every later test assumes a known baseline: a known board, a known image and known helper commands. If a test fails and you have no baseline, you cannot tell whether your recipe broke something or it never worked. A00 creates *test zero*.

1. A dedicated build directory

Create a fresh build directory for the lab with your usual `bitbake-setup` flow. Keep `prj_1` untouched. In `conf/local.conf`, add the following *below* the existing `EXTRA_IMAGE_FEATURES ?= "debug-tweaks"` line:

```
MACHINE = "beaglebone-yocto"
DISTRO = "poky"
PACKAGE_CLASSES = "package_ipk"
EXTRA_IMAGE_FEATURES += "ssh-server-dropbear package-management"

# where your "upstream" git repositories live (used in SRC_URI from A02 on)
BBB_LAB_SRC = "/home/<you>/bbb-lab-src"
```

> **Order matters (your first second-order lesson).** `?=` only assigns if the variable is still unset when that line is parsed. If you put the `+=` line *above* the `?=` line, the `?=` never applies and you silently lose `debug-tweaks`, which provides the password-less root login every T-test relies on. Poky's default `PACKAGE_CLASSES` is `package_rpm`. This curriculum uses ipk and `opkg` throughout.

2. Host helpers

Save this as `~/bbb-lab-env.sh` and source it in every build shell, after the bitbake environment:

```
export SRC=$HOME/bbb-lab-src               # your upstream git repos
export LAYER=<path-to>/meta-bbb-lab        # created in A01
export BBB=192.168.1.50                    # the board's IP (see step 4)
export LED=beaglebone:green:usr3           # the LED you will drive; use a name from YOUR board
bbb() { ssh -o StrictHostKeyChecking=no root@"$BBB" "$@"; }        # run on the board
var() { bitbake-getvar -q -r "$1" --value "$2"; }                 # one variable of one recipe
```

`bitbake-getvar` without `--value` prints the variable's full **history**: every file and line that touched it. Use that form whenever a value surprises you.

Paths in the tests such as `tmp/deploy/…` and `tmp/sysroots-components/…` are relative to your build directory and assume the default `TMPDIR`. If your `bitbake-setup` layout puts `TMPDIR` or `DL_DIR` elsewhere, check them with `bitbake-getvar -q --value TMPDIR` and adjust.

3. Baseline image, flash, boot

```
bitbake core-image-minimal
ls tmp/deploy/images/beaglebone-yocto/ | grep 'core-image-minimal.*wic'
sudo bmaptool copy tmp/deploy/images/beaglebone-yocto/core-image-minimal-beaglebone-yocto.rootfs.wic /dev/sdX
```

The `beaglebone-yocto` machine adds `wic` and `wic.bmap` to `IMAGE_FSTYPES` and uses `beaglebone-yocto.wks`: a 32 MiB vfat boot partition (MLO, `u-boot.img`, `zImage`, the three `.dtb` files) followed by an ext4 root. To boot from SD, hold the **S2** button while you apply power if the eMMC still holds a bootloader. The serial console runs at 115200 8N1 on the 6-pin J1 header (`ttyS0`).

4. Record the baseline (test zero)

On the board (serial console first, then find the IP with `ip addr show eth0`), run the following and save the output to `baseline.txt` in your lab notebook:

```
uname -r
cat /proc/device-tree/model; echo
ls /sys/class/leds/
cat /sys/class/leds/*/max_brightness
for c in i2cdetect devmem; do command -v $c || echo "$c: absent"; done
opkg --version | head -1
```

> **Expect.** The model is `TI AM335x BeagleBone Black`. There are four user LEDs; set `$LED` to one of them. On mainline DTS the fourth is usually `beaglebone:green:usr3`, but use **your** names. `i2cdetect` and `devmem` are absent, because poky's busybox `defconfig` disables both (you change that in A10). `opkg` runs, because of `package-management`.

5. The fast deploy loop (A01–A11)

Reflashing after every change is slow. Until A12 gives you a proper package feed, deploy single packages:

```
bitbake bbbctl
ls tmp/deploy/ipk/                       # cortexa8hf-neon/, beaglebone_yocto/, all/
scp tmp/deploy/ipk/cortexa8hf-neon/bbbctl_*.ipk root@$BBB:/tmp/
bbb opkg install --force-reinstall /tmp/bbbctl_*.ipk
```

Copy the dependencies too, or reflash when a package gains a new dependency. In A12 you replace this loop with `opkg upgrade` from a feed, and you will see why the feed is better.

> **Done when** the board boots your `core-image-minimal`, `ssh root@$BBB` works without a password, and `baseline.txt` is saved.

PHASE 1 **Recipe foundations**

A01 · Your layer and your first recipe

You build	`bbb-hello`: one C file, no build system
New concept	layer anatomy, FILESPATH lookup, `S` / `B` / `D`, the cross `CC`, licensing
Builds on	A00
Activation energy	Low. Expect friction only on `LIC_FILES_CHKSUM` and on where files land in `WORKDIR`
Ladder link	makedevs: a recipe with no class beyond `base`

Why this exists (Chesterton's Fence). A recipe is a contract that answers four questions: *where is the source, how is it built, where does it install, under what license*. Before any class does this for you, write all four answers by hand once. Then every class you meet later is just a pre-written answer to one of the four.

What you build

`bbb-hello.c` prints three lines and exits 0:

```
bbb-hello 1.0
board: TI AM335x BeagleBone Black
arch: armv7l
```

`board` is read from `/proc/device-tree/model` (strip the trailing NUL). `arch` comes from `uname(2)` (`.machine`). The version string `1.0` must come from the recipe at compile time (`-DBBB_HELLO_VERSION=...`), not from the C file. You will see why in A02.

What you write

- Create the layer with `bitbake-layers create-layer $LAYER`, then add it with `bitbake-layers add-layer`. Open `conf/layer.conf` and explain every line in your notebook: `BBFILE_COLLECTIONS`, `BBFILE_PATTERN`, `BBFILE_PRIORITY`, `LAYERDEPENDS`, `LAYERSERIES_COMPAT`. Delete the generated `example` recipe.
- `recipes-bbblab/bbb-hello/bbb-hello_1.0.bb`, with the source in `recipes-bbblab/bbb-hello/files/bbb-hello.c`.
- License: MIT, using the common-license text: `file://${COMMON_LICENSE_DIR}/MIT;md5=0835ade698e0bcf8506ecda2f7b4f302`.
- `SRC_URI` with `file://`. On scarthgap, local files unpack into `${WORKDIR}`, so set `S = "${WORKDIR}"`. (On styhead and later this changes to `${UNPACKDIR}`. Your tree is scarthgap.)
- Write `do_compile` yourself using `${CC} ${CFLAGS} ${LDFLAGS}`. Note that `meta-skeleton`'s `hello_1.0.bb` uses only `${CC} ${LDFLAGS}`. Decide whether dropping `CFLAGS` matters and write the answer down.
- Write `do_install` yourself: `install -d ${D}${bindir}`, then `install -m 0755`.
- Add it to the image *temporarily* in `local.conf`: `IMAGE_INSTALL:append = " bbb-hello"`. Note the leading space and why `:append` is needed instead of `+=` here.

What you learn

- How BitBake turns `bbb-hello_1.0.bb` into `PN`, `PV` and a `WORKDIR` (`tmp/work/cortexa8hf-neon-poky-linux-gnueabi/bbb-hello/1.0/`. Since 4.3, PR is no longer part of the path).
- How `file://` is resolved through `FILESPATH`: `${BP}`, `${BPN}` and `files/` directories, each tried with override subdirectories first. This becomes important in A10.
- What `${CC}` really contains: the cross prefix, `-mfloat-abi=hard`, `-mcpu=cortex-a8` and `--sysroot=`. This is why you never call `arm-…-gcc` yourself.
- The default `PACKAGES` / `FILES` split, and why one binary still produces `-dbg` and `-src` packages.

Tests

A01-H1 **Your layer is registered**

```
bitbake-layers show-layers | grep meta-bbb-lab
grep -E 'LAYERSERIES_COMPAT|BBFILE_PRIORITY' $LAYER/conf/layer.conf
```

Expect your layer path with its priority, and `LAYERSERIES_COMPAT_… = "scarthgap"`.

A01-H2 **Clean build, no QA noise**

```
bitbake bbb-hello 2>&1 | tee /tmp/a01.log | grep -E 'WARNING|ERROR' ; echo "count=$?"
```

Expect no lines printed (`count=1` because grep found nothing).

A01-H3 **It is an ARM hard-float binary**

```
file $(var bbb-hello D)/usr/bin/bbb-hello
```

Expect `ELF 32-bit LSB … ARM, EABI5 …, interpreter /lib/ld-linux-armhf.so.3`.

A01-H4 **What `CC` actually is**

```
var bbb-hello CC | tr ' ' '\n' | grep -E 'sysroot|float-abi|mcpu|gcc'
```

Expect an `arm-poky-linux-gnueabi-gcc` line, `-mfloat-abi=hard`, a CPU/arch flag and `--sysroot=…/recipe-sysroot`.

A01-H5 **Packaging**

```
oe-pkgdata-util list-pkgs -p bbb-hello
oe-pkgdata-util list-pkg-files bbb-hello
```

Expect at least `bbb-hello` and `bbb-hello-dbg`. `/usr/bin/bbb-hello` is in `bbb-hello`.

A01-T1 **Runs on the board**

```
bbb bbb-hello
```

Expect exactly the three lines in the specification, with the version from the recipe.

Break it (Inversion)

A01-B1 **Bypass `${CC}`.** Replace `${CC}` with `${TARGET_PREFIX}gcc` in `do_compile`. *Predict* the error before you build.

Expect a missing system header (`stdio.h`). Explain with A01-H4 which part of `CC` you removed, and why OE builds its cross gcc so that a missing `--sysroot` fails loudly instead of falling back to your host headers.

A01-B2 **Install something you don't ship.** Also install a notes file to `${D}/opt/bbb-hello/notes.txt`. *Predict* which task fails and what the message says.

Expect `do_package_qa` fails with **installed-vs-shipped** (`/opt` is in no package's `FILES`). Fix it with `FILES:${PN} += "/opt/bbb-hello"`, then explain why `${datadir}/bbb-hello` would *not* have needed a fix.

A01-B3 **Wrong license checksum.** Change one hex digit of the md5.

Expect a **license-checksum** error that prints the *correct* md5 and the license text. This is also how you will get the md5 for your own `LICENSE` files from A02 on.

Done when H1–H5 and T1 pass, and the notebook has your three predictions next to the actual errors.

A02 · An upstream project: bbbctl 0.1 from git

You build	`bbbctl` 0.1: multi-file C CLI with a conventional `Makefile`
New concept	the git fetcher, `SRCREV`, `PV = "x+git"`, `oe_runmake`, upstream install conventions
Builds on	A01
Activation energy	Low. The trap is a Makefile that hard-codes `CC`
Ladder link	libaio: Makefile-only build

> **Why this exists.** Real software is not a file in your layer. It is someone else's repository at a specific revision, built with *their* build system. The recipe stops being the build script and becomes an **adapter** between upstream conventions (`CC`, `DESTDIR`, `PREFIX`) and OE's variables. A01 was the build script. A02 is the adapter.

What you build

Create the repository `$SRC/bbbctl` (`git init -b main`) containing `main.c`, `led.c`, `led.h`, `Makefile`, `LICENSE` (MIT text), and optionally `bbbctl.1`.

```
bbbctl --version              ->  bbbctl 0.1
bbbctl led list               ->  one LED name per line, from /sys/class/leds
bbbctl led <name> on|off      ->  sets trigger to "none", writes max_brightness or 0
```

The Makefile must follow upstream conventions:

- `CC ?= gcc`, with `CFLAGS` and `LDFLAGS` taken from the environment. **Never strip.**
- An `install` target that honours `DESTDIR` and `PREFIX` (default `/usr/local`) and installs `$(DESTDIR)$(PREFIX)/bin/bbbctl` and the man page.

Commit, then tag `v0.1`.

What you write

- `bbbctl_0.1.bb` with `SRC_URI = "git://${BBB_LAB_SRC}/bbbctl;protocol=file;branch=main"` and `S = "${WORKDIR}/git"`.
- `SRCREV` pinned to the commit of `v0.1`. Set `PV = "0.1+git"`. Since 4.3 you no longer write `${SRCPV}`, because the `+` makes BitBake append the revision to **PKGV** at packaging time.
- `LIC_FILES_CHKSUM` pointing at your repository's `LICENSE`. Get the md5 the A01-B3 way.
- `do_compile` with `oe_runmake`, and `do_install` with `oe_runmake install DESTDIR=${D} PREFIX=${prefix}`.

What you learn

- What `do_fetch` stores in `DL_DIR/git2/` (a bare mirror) and what `do_unpack` checks out into `${WORKDIR}/git`.
- `SRCREV` pinning vs `${AUTOREV}`, and why a pinned revision is what makes a build reproducible.
- `PV` vs `PKGV`: what the package version actually looks like on the target.
- `?=` vs `=` in a Makefile, and why OE *exports* `CC` instead of passing it on the command line.
- The automatic `${PN}-doc` split for `${mandir}`.

Tests

A02-H1 The fetcher mirror exists

```
ls $(var bbbctl DL_DIR)/git2/ | grep -i bbbctl
```

Expect a mirror directory named after your repository path.

A02-H2 Package versioning

```
oe-pkgdata-util read-value PV,PKGV bbbctl
git -C $SRC/bbbctl rev-parse --short=10 v0.1
```

Expect `PV` is `0.1+git`. `PKGV` begins with `0.1+git` and contains the start of your `v0.1` commit hash.

A02-H3 Docs split

```
oe-pkgdata-util list-pkg-files -p bbbctl
```

Expect `/usr/bin/bbbctl` in `bbbctl`. The man page (if you wrote one) is in `bbbctl-doc` and **not** in `bbbctl`.

A02-T1 **Behaviour**

```
bbb bbbctl --version
bbb bbbctl led list
bbb "bbbctl led $LED on;  cat /sys/class/leds/$LED/brightness /sys/class/leds/$LED/max_brightness"
bbb "bbbctl led $LED off; cat /sys/class/leds/$LED/brightness"
```

> **Expect** `bbbctl 0.1`. The list matches `baseline.txt`. After `on`, brightness equals `max_brightness` and you can see the LED lit. After `off`, brightness is `0`.

Break it (Inversion)

A02-B1 **The hostile Makefile.** In upstream, change `CC ?= gcc` to `CC = gcc`, commit, and bump `SRCREV`. *Predict*: which compiler runs, which task fails first, and why.

> **Expect** your *host* gcc builds an x86-64 binary. The build fails afterwards, either in `do_package` (the ARM `objcopy` / `strip` cannot handle an x86-64 ELF) or in `do_package_qa` with **arch** (`Architecture did not match (x86-64, expected ARM)`). Read the log and note which one came first. Fix it **without** touching upstream by setting `EXTRA_OEMAKE = "CC='${CC}'"`, then revert upstream. Write down when you would carry a recipe-side override and when you would push the fix upstream.

A02-B2 **The pinned revision.** Commit a visible change upstream (for example a new `--version` string) but leave `SRCREV` alone. Rebuild. *Predict*.

> **Expect** nothing changes. Then try `SRCREV = "${AUTOREV}"`, rebuild, and observe the change. Restore the pin and write one sentence on what AUTOREV costs you.

> **Done when** H1–H3 and T1 pass, and you can explain what B1 showed about upstream conventions.

A03 · A shared library from scratch: libbbbhw 0.1

You build	`libbbbhw.so.1`: the hardware helper library that every later app uses
New concept	SONAME/real name/linker name, library package split, sysroot staging, debian renaming
Builds on	A02
Activation energy	Medium. Three files that look alike (`.so`, `.so.1`, `.so.1.0.0`) each end up in a different place
Ladder link	libaio (Makefile library), acl (`libacl` vs `acl-dev`)

> **Why this exists.** A library has two audiences: **programs at runtime** need the versioned shared object, and **compilers at build time** need the headers, the unversioned `.so` symlink and the `.pc` file. OE serves those two audiences with two separate mechanisms, *packages* (runtime) and the *sysroot* (build time). A library recipe is the first place where the two clearly diverge.

What you build

Create the repository `$SRC/libbbbhw` with the header `bbbhw.h`:

```
#define BBBHW_VERSION "0.1"
const char *bbbhw_version(void);                        /* "0.1" */
int  bbbhw_board_model(char *buf, size_t len);          /* /proc/device-tree/model; 0 or -errno */
int  bbbhw_led_list(char names[][64], int max);         /* count or -errno */
int  bbbhw_led_set(const char *led, int on);            /* trigger=none; brightness=max or 0 */
int  bbbhw_parse_pattern(const char *s, unsigned *on_ms, unsigned *off_ms); /* "100,900" -> 0, else -EINVAL */
```

Makefile requirements:

- Compile with `-fPIC`. Link `libbbbhw.so.1.0.0` with `-shared -Wl,-soname,libbbbhw.so.1`.
- Install the real file, then symlinks `libbbbhw.so.1 → libbbbhw.so.1.0.0` and `libbbbhw.so → libbbbhw.so.1`.
- Install `bbbhw.h` to `$(INCLUDEDIR)`, and generate `bbbhw.pc` from a `bbbhw.pc.in` template into `$(LIBDIR)/pkgconfig`.
- `LIBDIR ?= $(PREFIX)/lib`. The recipe passes `LIBDIR=${libdir}`.
- The `.pc` file must contain `prefix=/usr`, not a build path.

What you write

`libbbbhw_0.1.bb`. There are no new recipe variables beyond A02. The point is what OE *does* with standard install locations. Add `libbbbhw` to `IMAGE_INSTALL` for T1.

What you learn

- Default `PACKAGES` order: `${PN}-src ${PN}-dbg ${PN}-staticdev ${PN}-dev ${PN}-doc ${PN}-locale … ${PN}`. **The first package whose `FILES` matches a file takes it**, which is why `-dev` gets the `.so` symlink before `${PN}` is ever considered.
- `SOLIBS` vs `FILES_SOLIBSDEV`.
- `do_populate_sysroot`: what is staged (`SYSROOT_DIRS`, such as headers, libs and `.pc` files) and what is not (`/usr/bin`). This shows up again in A07.
- The `debian` class (from `INHERIT_DISTRO`) renames `libbbbhw` to `libbbbhw1` using the SONAME. Recipe-space and runtime package names diverge here for the first time.
- `pkgdata` shlibs: how OE records that `libbbbhw1` *provides* `libbbbhw.so.1`.

Tests

A03-H1 SONAME is set

```
readelf -d $(var libbbbhw D)/usr/lib/libbbbhw.so.1.0.0 | grep SONAME
```

Expect `Library soname: [libbbbhw.so.1]`.

A03-H2 The split

```
oe-pkgdata-util list-pkg-files libbbbhw libbbbhw-dev
```

Expect `.so.1` and `.so.1.0.0` are in `libbbbhw`. `.so`, `bbbhw.h` and `pkgconfig/bbbhw.pc` are in `libbbbhw-dev`.

A03-H3 Renaming

```
bitbake-getvar -q INHERIT_DISTRO
oe-pkgdata-util lookup-pkg libbbbhw libbbbhw-dev
oe-pkgdata-util list-pkgs -r -p libbbbhw
```

> **Expect** `INHERIT_DISTRO` contains `debian`. The runtime name of `libbbbhw` is `libbbbhw1`. Record what `-dev` maps to.

A03-H4 What reached the sysroot (and what did not)

```
find tmp/sysroots-components/cortexa8hf-neon/libbbbhw -type f -o -type l | sort
```

> **Expect** header, `.pc` and all three library names. Nothing under `usr/bin` (there is nothing to stage, but remember this directory list for A07).

A03-H5 The `.pc` is clean

```
cat tmp/sysroots-components/cortexa8hf-neon/libbbbhw/usr/lib/pkgconfig/bbbhw.pc
```

> **Expect** `prefix=/usr` and no `tmp/` path anywhere. If one were present, `do_populate_sysroot` QA (**pkgconfig**) would have failed.

A03-T1 Runtime footprint

```
bbb 'ls -l /usr/lib/libbbbhw*; opkg list-installed | grep -i bbbhw'
```

> **Expect** only `libbbbhw.so.1` and `libbbbhw.so.1.0.0` on the target, no bare `.so`, and the package is listed as `libbbbhw1`.

Break it (Inversion)

A03-B1 Fight the split. First add `FILES:${PN} += "${libdir}/libbbbhw.so"` and nothing else. *Predict* where the `.so` ends up. Then also set `FILES_SOLIBSDEV = ""`. *Predict* again.

> **Expect** (1) the `.so` stays in `-dev`, because first match wins and `-dev` comes earlier in `PACKAGES`. (2) It moves to `${PN}`, and `do_package_qa` fails with **dev-so**: `non -dev/-dbg/nativesdk- package libbbbhw contains symlink .so`. Revert both changes.

Done when H1–H5 and T1 pass, and you can explain in two sentences why the bare `.so` belongs to `-dev`.

A04 · Consuming the library: bbbctl 0.2

You build	`bbbctl` 0.2, which drops its own `led.c` and links `libbbbhw`
New concept	`DEPENDS`, recipe-specific sysroots, pkg-config in cross builds, automatic shlib `RDEPENDS`, `PREFERRED_VERSION`
Builds on	A03
Activation energy	Medium. The idea that "it is built, but my recipe cannot see it" takes a moment
Ladder link	acl/attr: build-time vs run-time dependency

> **Why this exists.** Since Yocto 2.3, every recipe gets a **private sysroot** that contains only what its `DEPENDS` chain stages. This is what makes builds deterministic. Without it, a library that happened to be built earlier would leak into recipes that never declared it. You declare build-time needs (`DEPENDS`), and OE works out the run-time ones (`RDEPENDS`) from the ELF files.

What you build

`bbbctl` 0.2 keeps the 0.1 commands and adds `bbbctl board`, which prints the model via `bbbhw_board_model()`. `--version` prints `bbbctl 0.2 (libbbbhw 0.1)`, with the library version taken at **runtime** from `bbbhw_version()`. The Makefile gets the compile and link flags from `$(PKG_CONFIG) --cflags/--libs bbbhw`, with `PKG_CONFIG ?= pkg-config`. Tag `v0.2`.

What you write

- A new recipe file `bbbctl_0.2.bb`. **Keep `bbbctl_0.1.bb` for now.**
- `DEPENDS = "libbbbhw"` and `inherit pkgconfig`.
- Remove `libbbbhw` from `IMAGE_INSTALL`. Only `bbbctl` stays listed.

What you learn

- `do_prepare_recipe_sysroot` and what `recipe-sysroot/` contains, and why it differs from recipe to recipe.
- How the pkg-config wrapper rewrites paths with `PKG_CONFIG_SYSROOT_DIR`.
- shlibs: OE reads `NEEDED` entries from your ELF, maps them to the providers recorded in A03, and writes `RDEPENDS` for you.
- How two versions of one recipe coexist: the highest wins unless `PREFERRED_VERSION_bbbctl` says otherwise.

Tests

A04-H1 The sysroot contains your library, and only by declaration

```
ls $(var bbbctl RECIPE_SYSROOT)/usr/include/bbbhw.h $(var bbbctl RECIPE_SYSROOT)/usr/lib/libbbbhw.so*
```

Expect all present.

A04-H2 The task edge

```
bitbake -g bbbctl && grep '"bbbctl.do_prepare_recipe_sysroot" -> "libbbbhw.do_populate_sysroot"' task-depends.dot
```

Expect exactly one matching edge.

A04-H3 Automatic run-time dependency

```
readelf -d $(var bbbctl D)/usr/bin/bbbctl | grep NEEDED
oe-pkgdata-util read-value RDEPENDS bbbctl
```

Expect `NEEDED` includes `libbbbhw.so.1`. `RDEPENDS` contains `libbbbhw1`, and you did not write it. Note whether a version constraint appears and where it comes from.

A04-H4 Two versions, one choice

```
bitbake-layers show-recipes bbbctl
```

Expect `0.1+git` and `0.2+git`, with 0.2 selected. Set `PREFERRED_VERSION_bbbctl = "0.1+git"` in `local.conf`, rerun, observe 0.1 being selected, then remove the line.

A04-T1 The library arrives without being asked for

```
bbb 'opkg list-installed | grep -E "bbbctl|bbbhw"; bbbctl --version; bbbctl board'
```

Expect both `bbbctl` and `libbbbhw1` are installed. Version line `bbbctl 0.2 (libbbbhw 0.1)`. Board line `TI AM335x BeagleBone Black`.

Break it (Inversion)

A04-B1 **Undeclared dependency.** Remove `DEPENDS`, then `bitbake -c cleansstate bbbctl && bitbake bbbctl`. *Predict*: libbbbhw is already built, so does the build succeed?

> **Expect** no. pkg-config reports `Package bbbhw was not found`, and/or `bbbhw.h: No such file or directory`. Explain this with A04-H1. Restore `DEPENDS`.

A04-B2 **The missing SONAME.** In upstream libbbbhw, drop `-Wl,-soname,…`, rebuild both recipes, and look at A04-H3 again. *Predict* what `NEEDED` now says and what that would mean on a target that does not have `libbbbhw-dev` installed.

> **Expect** `NEEDED: libbbbhw.so`, the *linker* name, which only `-dev` ships. That binary needs a development package at runtime, which is the bug A03's three-name scheme prevents. Restore the SONAME.

Done when H1–H4 and T1 pass, `bbbctl_0.1.bb` is deleted, and B2's consequence is written in your own words.

PHASE 2 Build systems & configurability

A05 · Autotools from scratch: bbbinfo 1.0

You build	`bbbinfo` 1.0, a system report tool, with `configure.ac` + `Makefile.am` written by you
New concept	autotools.bbclass in practice: `autoreconf`, cross `configure`, `EXTRA_OECONF`, `B ≠ S`, configure-time QA
Builds on	A04 (pkg-config, `DEPENDS libbbbhw`)
Activation energy	Medium, but lower than acl was. You studied `autotools.bbclass` from the class side; now you write the input it consumes
Ladder link	which (first `inherit autotools`), acl (autoreconf, `EXTRA_OECONF`)

Why this exists. acl cost you two days because you met Autotools and its OE class at the same time. Here you only *write* the Autotools input, the two files upstream authors write. Once you have produced a `configure.ac`, every Autotools recipe reduces to three questions: which `--enable/--with` options, which `DEPENDS`, and how it is split.

What you build

Create the repository `$SRC/bbbinfo`. `bbbinfo` prints:

```
model: TI AM335x BeagleBone Black          (via bbbhw_board_model)
kernel: <uname -r>
cpu: <"model name" field of /proc/cpuinfo>
memtotal_kb: <MemTotal of /proc/meminfo>
label: <compile-time label, default "unnamed">
libbbbhw: <bbbhw_version()>
```

`configure.ac` must use, at minimum: `AC_INIT([bbbinfo],[1.0])`, `AM_INIT_AUTOMAKE([foreign])`, `AC_PROG_CC`, `PKG_CHECK_MODULES([BBBHW], [bbbhw >= 0.1])`, `AC_ARG_WITH([board-label],…)` feeding `AC_DEFINE_UNQUOTED([BOARD_LABEL],…)`, `AC_CONFIG_HEADERS`, `AC_CONFIG_FILES([Makefile])`. `Makefile.am` uses `bin_PROGRAMS` with `_CFLAGS = $(BBBHW_CFLAGS)` and `_LDADD = $(BBBHW_LIBS)`.

Do not commit generated files (`configure`, `Makefile.in`, `aclocal.m4`). OE runs `autoreconf` itself, and that is the realistic case. Before writing the recipe, check on your PC that `autoreconf -fi && ./configure && make` works natively.

What you write

`bbbinfo_1.0.bb` with `inherit autotools pkgconfig`, `DEPENDS = "libbbbhw"`, and `EXTRA_OECONF = "--with-board-label=bbb-lab"`.

What you learn

- What `do_configure` really runs: `autoreconf` with OE's `aclocal` paths, then `configure` with `--build`, `--host=arm-poky-linux-gnueabi`, `--prefix` and the sysroot options (see `CONFIGUREOPTS`).
- Out-of-tree builds: autotools.bbclass sets `B = "${WORKDIR}/build"`, so `config.log` is in `${B}`, not `${S}`.
- Where `PKG_CHECK_MODULES` comes from (`pkg.m4`, staged by `pkgconfig-native`), and so why `inherit pkgconfig` is not optional.
- Two configure-time QA checks from `insane.bbclass`: **unknown-configure-option** and **configure-unsafe**.

Tests

A05-H1 Out-of-tree build

```
echo "S=$(var bbbinfo S)"; echo "B=$(var bbbinfo B)"; ls $(var bbbinfo B)/config.log
```

Expect `B` is `…/build`, different from `S`. `config.log` exists in `B`.

A05-H2 It was a cross configure with your option

```
grep -m1 -- '--host=' $(var bbbinfo B)/config.log
grep -c -- 'with-board-label=bbb-lab' $(var bbbinfo B)/config.log
grep -c 'is unsafe for cross-compilation' $(var bbbinfo B)/config.log
```

Expect `--host=arm-poky-linux-gnueabi`, a count of 1 or more, and a count of `0`.

A05-T1 Output

```
bbb bbbinfo; bbb uname -r
```

Expect `kernel:` matches `uname -r`. `cpu:` starts with `ARMv7 Processor`. `memtotal_kb` is a little under 524288 (512 MiB minus reservations). `label: bbb-lab`. `libbbbhw: 0.1`.

Break it (Inversion)

A05-B1 **An option configure doesn't know.** Add `--enable-frobnicate` to `EXTRA_OECONF`. *Predict* whether configure fails, and which task fails.

> **Expect** configure succeeds and only warns, but the `do_configure` QA post-function fails with **unknown-configure-option** (`configure was passed unrecognised options: --enable-frobnicate`). This is ERROR_QA on scarthgap.

A05-B2 **Drop `inherit pkgconfig`.** *Predict* which step fails and why, given that you never call `pkg-config` directly.

> **Expect** `autoreconf` / `configure` fails around `PKG_CHECK_MODULES` (an undefined-macro or syntax error), because `pkg.m4` never reached the recipe sysroot. Connect this to A04-H1: same mechanism, different file.

Done when H1, H2 and T1 pass, and you can recite what `--host`, `--build` and `--with-libtool-sysroot` mean without looking.

A06 · CMake: bbb-ledd 0.1

You build	`bbb-ledd` 0.1, a foreground LED blinker daemon, built with CMake
New concept	cmake.bbclass: the generated toolchain file, find-root-path rules, `EXTRA_OECMAKE`, Ninja
Builds on	A04 (libbbbhw via pkg-config)
Activation energy	Low to medium
Ladder link	expat (cmake class)

> **Why this exists.** Autotools learns about the target through command-line options. CMake learns through a **toolchain file**. OE generates one per recipe, and it decides what `find_*()` is allowed to see. Most CMake cross-build bugs are really `find_*()` searching the host. Here you see the fence that prevents that.

What you build

Create the repository `$SRC/bbb-ledd`:

```
bbb-ledd [-l LED] [-p ON,OFF] [-f] [--version]
  -l LED     LED to drive (default: compile-time BBB_LEDD_DEFAULT_LED)
  -p ON,OFF  pattern in ms, parsed with bbbhw_parse_pattern() (default 100,900)
  -f         foreground (0.1 is always foreground; accept the flag)
SIGTERM/SIGINT  -> LED off, exit status 0
```

`CMakeLists.txt` must use `include(GNUInstallDirs)`, `find_package(PkgConfig REQUIRED)`, `pkg_check_modules(BBBHW REQUIRED IMPORTED_TARGET bbbhw)`, a cache variable `BBB_LEDD_DEFAULT_LED` (STRING) passed as a compile definition, and install the binary to `${CMAKE_INSTALL_SBINDIR}`.

What you write

`bbb-ledd_0.1.bb` with `inherit cmake pkgconfig` and `DEPENDS = "libbbbhw"`. Add a recipe variable `BBB_LEDD_LED ?= "beaglebone:green:usr3"` and pass it with `EXTRA_OECMAKE = "-DBBB_LEDD_DEFAULT_LED=${BBB_LEDD_LED}"`. The `?=` lets `local.conf` override it per board without a bbappend.

What you learn

- `${WORKDIR}/toolchain.cmake`: `CMAKE_SYSROOT`, `CMAKE_FIND_ROOT_PATH`, and the `CMAKE_FIND_ROOT_PATH_MODE_*` settings (programs from the host side, libraries and headers only from the sysroot).
- `OECMAKE_GENERATOR` (Ninja), `${B}` vs `${S}`, and `CMakeCache.txt` as the record of what was decided.
- The `?=` recipe-variable idiom for user-tunable defaults.

Tests

A06-H1 The fence

```
grep -E 'CMAKE_SYSROOT|CMAKE_FIND_ROOT_PATH' $(var bbb-ledd WORKDIR)/toolchain.cmake
```

Expect paths inside this recipe's `recipe-sysroot`, and `MODE_PROGRAM` vs `MODE_LIBRARY/INCLUDE` set differently. Write down why.

A06-H2 Generator and cache

```
var bbb-ledd OECMAKE_GENERATOR; ls $(var bbb-ledd B)/build.ninja
grep BBB_LEDD_DEFAULT_LED $(var bbb-ledd B)/CMakeCache.txt
```

Expect `Ninja`, the file exists, and the cache holds your LED name.

A06-H3 Override from local.conf

Add `BBB_LEDD_LED:pn-bbb-ledd = "beaglebone:green:usr2"` to `local.conf`, rebuild, rerun H2, then remove the line.

Expect the cache value follows the override. Only `bbb-ledd` rebuilds.

A06-T1 Blinks and stops cleanly

```
# on the board
LED=beaglebone:green:usr3          # your LED
bbb-ledd -p 100,400 &
sleep 2
for i in $(seq 20); do cat /sys/class/leds/$LED/brightness; usleep 50000; done | uniq -c
kill $!; wait $!; echo "exit=$?"; cat /sys/class/leds/$LED/brightness
```

Expect alternating runs of max and `0` (roughly 2 samples on, 8 off), `exit=0`, and final brightness `0`.

Break it (Inversion)

A06-B1 **Reach for the host.** Add `include_directories(/usr/include)` to `CMakeLists.txt` and rebuild. *Predict* whether OE notices.

> **Expect** OE's gcc emits an `include location "/usr/include" is unsafe for cross-compilation` diagnostic. Find it in `log.do_compile` and record whether your build treats it as a warning or an error. Explain why OE patches gcc to do this at all.

A06-B2 **Drop `inherit pkgconfig`.** *Predict.*

> **Expect** CMake configure fails with `Could NOT find PkgConfig`. The build's `PATH` contains only `HOSTTOOLS` and the native sysroot, and `pkg-config` is in neither unless `pkgconfig-native` is staged.

Done when H1–H3 and T1 pass.

A07 · Native tools and build-time code generation: bbb-patc

You build	`bbb-patc`, a pattern compiler that runs **on your PC** during the build, and `bbb-ledd` 0.2 using its output
New concept	target vs native, `BBCLASSEXTEND = "native"`, `DEPENDS += "foo-native"`, native sysroot staging
Builds on	A06
Activation energy	Medium. "Same recipe, two architectures" takes a minute to absorb
Ladder link	none directly. Every `*-native` in a `DEPENDS` line you have read works this way

Why this exists. Builds often need to *run* a program: a code generator, a compressor, a signing tool. A program built for the target cannot run on the build host, and a host program from `/usr/bin` would make the build depend on whatever happens to be installed there. OE's answer is to build the tool itself, for the build host, as a `-native` recipe.

What you build

Create the repository `$SRC/bbb-patc` (C + Makefile, A02 conventions). It reads a pattern file:

```
# name: on off on off ...   (milliseconds)
heartbeat: 100 100 100 700
sos: 150 150 150 150 150 450 450 150 450 150 450 450 150 150 150 150 150 1050
```

and writes a C header with one `static const unsigned` array per pattern, plus a table `bbb_patterns[]` of `{name, count, steps}` and `#define BBB_PATTERN_COUNT`. Usage: `bbb-patc INPUT -o OUTPUT`. It exits 1 with a line number on an odd step count or a non-number.

`bbb-ledd` 0.2 adds `patterns.txt` to its repository, generates `patterns.h` in the CMake binary directory with `find_program(BBB_PATC bbb-patc REQUIRED)` + `add_custom_command`, and gains two options: `-P NAME` (use a built-in pattern) and `-L` (list built-in patterns).

What you write

- `bbb-patc_1.0.bb` with `BBCLASSEXTEND = "native"`. One recipe, two variants.
- In `bbb-ledd_0.2.bb` (rename the file and bump `SRCREV`): `DEPENDS += "bbb-patc-native"`.

What you learn

- What the `native` class changes: `WORKDIR` under `x86_64-linux`, the host compiler, and a prefix inside the native sysroot.
- `SYSROOT_DIRS` vs `SYSROOT_DIRS_NATIVE`: why `/usr/bin` is staged for native recipes and not for target ones. You saw the empty `usr/bin` in A03-H4.
- `RECIPE_SYSROOT_NATIVE/usr/bin` is on `PATH` during the build. `HOSTTOOLS` is the only other thing there.

Tests

A07-H1 Same recipe, two binaries

```
bitbake bbb-patc bbb-patc-native
file $(var bbb-patc D)/usr/bin/bbb-patc
find tmp/sysroots-components/x86_64/bbb-patc-native -name bbb-patc -exec file {} \;
var bbb-patc-native WORKDIR
```

Expect the first is ARM, the second is x86-64, and the native `WORKDIR` contains `x86_64-linux`.

A07-H2 The consumer sees the tool

```
ls $(var bbb-ledd RECIPE_SYSROOT_NATIVE)/usr/bin/bbb-patc
grep -c BBB_PATTERN_COUNT $(var bbb-ledd B)/patterns.h
```

Expect present, and a count of 1 or more.

A07-T1 Built-in patterns

```
bbb bbb-ledd -L
bbb 'bbb-ledd -P sos & sleep 8; kill $!'
```

Expect `heartbeat` and `sos` are listed, and you can see SOS on your LED.

Break it (Inversion)

A07-B1 **Depend on the target variant.** Change `bbb-patc-native` to `bbb-patc` in `DEPENDS`. *Predict* the failure and its exact cause. (There are two independent reasons this cannot work. Name both.)

> **Expect** CMake configure fails at `find_program(... REQUIRED)` because the tool is not found. Reason 1: target recipes do not stage `/usr/bin` into the sysroot. Reason 2: even if they did, an ARM ELF cannot execute on the build host.

A07-B2 **Change the tool, watch the consumer.** Change only a comment in `bbb-patc.c`, bump its `SRCREV`, and run `bitbake bbb-ledd`. *Predict*: does `bbb-ledd` recompile?

> **Expect** `bbb-patc-native` rebuilds. Whether `bbb-ledd` re-runs depends on whether the native tool's *output* changed: poky enables hash equivalence (`BB_SIGNATURE_HANDLER = "OEEquivHash"`), which can stop the rebuild cascade. Record what happened and justify it from the task list bitbake printed.

Done when H1, H2 and T1 pass, and B1's two reasons are in your notebook.

A08 · Optional features: PACKAGECONFIG and bbbinfo 1.1

You build	`bbbinfo` 1.1 with optional JSON output via `json-c` (OE-core, 0.17 on scarthgap)
New concept	`PACKAGECONFIG` and its fields, `??=` defaults, task signatures and `bitbake-diffsigs`
Builds on	A05
Activation energy	Low
Ladder link	acl: the coreutils `DISTRO_FEATURES` → `PACKAGECONFIG` → `DEPENDS` chain you traced

> **Why this exists.** A feature is a bundle: a configure flag, a build-time dependency and possibly a runtime one. If you keep them in three separate variables, they drift apart. `PACKAGECONFIG` turns the bundle into a single switch, and the signature system then proves exactly what that switch changed.

What you build

`bbbinfo` 1.1 adds `--json`. With `--with-json`, `configure` uses `PKG_CHECK_MODULES([JSONC],[json-c])` and defines `HAVE_JSON`, and `--json` prints one JSON object with the same six keys as the text output. With `--without-json`, `bbbinfo --json` prints `bbbinfo: built without JSON support` to stderr and exits **2**.

What you write

```
PACKAGECONFIG ??= "json"
PACKAGECONFIG[json] = "--with-json,--without-json,json-c"
```

Write down what the remaining positional fields of a `PACKAGECONFIG[...]` flag mean (RDEPENDS, RRECOMMENDS, conflicts), and why `??=` is used here rather than `?=`.

What you learn

- How `PACKAGECONFIG` expands into `EXTRA_OECONF` and `DEPENDS` at parse time.
- Switching features from outside the recipe: `PACKAGECONFIG:remove:pn-bbbinfo = "json"`.
- `bitbake-diffsigs` and `bitbake-dumpsig`: why a task re-ran, as a diff.
- The `PACKAGECONFIG ??= "${@bb.utils.filter('DISTRO_FEATURES', 'systemd', d)}"` idiom, which you use in A09.

Tests

A08-H1 Feature on (default)

```
var bbbinfo DEPENDS | tr ' ' '\n' | grep json
oe-pkgdata-util read-value RDEPENDS bbbinfo
```

> **Expect** `json-c` in `DEPENDS`, and a `libjson-c…` runtime dependency you never wrote.

A08-H2 Feature off, and *why* it rebuilt

Add `PACKAGECONFIG:remove:pn-bbbinfo = "json"` to `local.conf`, then:

```
bitbake bbbinfo && bitbake-diffsigs -t bbbinfo do_configure
oe-pkgdata-util read-value RDEPENDS bbbinfo
```

> **Expect** the diff names the changed variables (`PACKAGECONFIG` / `EXTRA_OECONF`, and the dependency change). `RDEPENDS` no longer lists `libjson-c…`.

A08-T1 JSON on (run this before H2 switches the feature off)

```
bbb bbbinfo --json | python3 -m json.tool
```

> **Expect** valid JSON (validated on your PC) with `model`, `kernel`, `cpu`, `memtotal_kb`, `label` and `libbbbhw`.

A08-T2 JSON off

```
bbb 'bbbinfo --json; echo rc=$?'
```

> **Expect** the “built without JSON support” message and `rc=2`.

Break it (Inversion)

A08-B1 **A feature that doesn't exist.** Add `PACKAGECONFIG:append:pn-bbbinfo = " xml"`. *Predict*: error, warning, or silence?

> **Expect** a **invalid-packageconfig** warning from `do_configure` QA (`bbbinfo: invalid PACKAGECONFIG: xml`). It is a warning, not an error, on scarthgap. Decide whether you would promote it to an error in your distro.

A08-B2 **Unbundle it.** Delete both `PACKAGECONFIG` lines and put `--with-json` straight into `EXTRA_OECONF`. *Predict*.

> **Expect** `configure` fails because `json-c` is not found, since nothing put it in the recipe sysroot. Same lesson as A04-B1, reached from a different direction.

Done when H1, H2, T1 and T2 pass (reflash or reinstall between T1 and T2), and `PACKAGECONFIG` is back to the default.

A09 · Services and system users: bbb-ledd 0.3

You build	`bbb-ledd` 0.3, started at boot, running as an unprivileged user, with a conffile and a state directory
New concept	update-rc.d.bbclass, systemd.bbclass, useradd.bbclass + pseudo ownership, `CONFFILES`
Builds on	A07 (bbb-ledd), A08 (DISTRO_FEATURES-driven PACKAGECONFIG)
Activation energy	High. Four classes meet in one recipe, and the privilege-drop order is easy to get wrong
Ladder link	psplash (update-rc.d, systemd)

> **Why this exists.** On a product, nobody types `bbb-ledd &`. Starting at boot, running with minimal privileges and keeping the user's configuration across upgrades are **packaging** problems, because they must be correct on a rootfs built on another machine. That is why users are created by `useradd.bbclass` at rootfs time, not by a script at first boot. A read-only rootfs (A19) would not allow the latter.

What you build

`bbb-ledd` 0.3 adds:

- `-u USER`: open the LED's `trigger` and `brightness` files **first**, then drop privileges (`setgroups`, `setgid`, `setuid`).
- `-s DIR` (default `/var/lib/bbb-ledd`): write `DIR/state` with `pattern=…`, `pid=…`, `started=<epoch>`.
- With the CMake option `WITH_SYSTEMD=ON`: link `libsystemd` and call `sd_notify(0, "READY=1")` after setup.
- In the repository: `bbb-ledd.init` (sysvinit, uses busybox `start-stop-daemon`, reads `/etc/default/bbb-ledd`), `bbb-ledd.service` (`Type=notify`, `EnvironmentFile=/etc/default/bbb-ledd`), and `bbb-ledd.default` containing `BBB_LEDD_ARGS="-l <your LED> -P heartbeat -u bbbled"`.

What you write

`bbb-ledd_0.3.bb`:

- `inherit cmake pkgconfig update-rc.d systemd useradd`
- `INITSCRIPT_NAME`, `INITSCRIPT_PARAMS = "defaults 90 10"`, `SYSTEMD_SERVICE:${PN}`
- `USERADD_PACKAGES = "${PN}"` and `USERADD_PARAM:${PN}` creating a system user `bbbled` with its own group, no login shell and no home creation.
- `do_install:append`: install the init script, the unit file and `/etc/default/bbb-ledd`, then create `${localstatedir}/lib/bbb-ledd` **owned by `bbbled`** with `install -d -o bbbled -g bbbled`.
- `CONFFILES:${PN} = "${sysconfdir}/default/bbb-ledd"`
- `PACKAGECONFIG ??= "${@bb.utils.filter('DISTRO_FEATURES', 'systemd', d)}"` and `PACKAGECONFIG[systemd] = "-DWITH_SYSTEMD=ON,-DWITH_SYSTEMD=OFF,systemd"`

Poky's default `INIT_MANAGER` is `sysvinit`, so this assignment is tested on sysvinit. The systemd half is built now and **runtime-tested in A18**, when your own distro switches to systemd.

What you learn

- What update-rc.d and useradd *generate*: maintainer scripts in the package, not code in your recipe.
- **pseudo**: why `chown bbbled` works inside `do_install` without root, and where the user database it consults comes from.
- `CONFFILES` and how opkg treats a modified configuration file on upgrade.
- How one recipe serves both init systems, and which class removes the files the other one does not need.

Tests

`A09-H1` Package contents

```
oe-pkgdata-util list-pkg-files bbb-ledd
```

> **Expect** `/usr/sbin/bbb-ledd`, `/etc/init.d/bbb-ledd`, `/etc/default/bbb-ledd` and `/var/lib/bbb-ledd`. On poky (sysvinit), check whether the systemd unit is shipped and explain the answer.

A09-H2 **Inside the .ipk** (scarthgap ipks use zstd, so install `zstd` on the host)

```
IPK=$(ls $PWD/tmp/deploy/ipk/cortexa8hf-neon/bbb-ledd_*.ipk | tail -1)
rm -rf /tmp/ipk && mkdir /tmp/ipk && cd /tmp/ipk && ar x $IPK && tar xf control.tar.zst
grep -l -E 'useradd|groupadd' preinst postinst 2>/dev/null; grep update-rc.d postinst
tar tvf data.tar.zst | grep var/lib/bbb-ledd; cd -
```

> **Expect** user creation code in a maintainer script, `update-rc.d` in `postinst`, and `var/lib/bbb-ledd` owned by `bbbled/bbbled` *inside the archive*.

A09-T1 **Started at boot, unprivileged**

```
bbb 'ps | grep "[b]bb-ledd"; id bbbled; ls -ld /var/lib/bbb-ledd; cat /var/lib/bbb-ledd/state'
```

> **Expect** the process runs as `bbbled`. The uid is in the system range (below 1000). The directory is owned by `bbbled`. The state file shows `pattern=heartbeat`, and the LED shows the heartbeat pattern.

A09-T2 **Runlevels**

```
bbb 'ls /etc/rc5.d | grep bbb-ledd; /etc/init.d/bbb-ledd stop; sleep 1; ps | grep -c "[b]bb-ledd"; /etc/init.d/bbb-ledd start'
```

> **Expect** `S90bbb-ledd`. After `stop`, a count of `0` and the LED off. After `start`, the pattern resumes.

A09-T3 **Conffile survives reinstall**

Edit `/etc/default/bbb-ledd` on the board (`-P sos`), then copy and `opkg install --force-reinstall` the same ipk.

> **Expect** your edit survives. Find where opkg put the package's copy, if it kept one.

Break it (Inversion)

A09-B1 **Remove `inherit useradd`** (keep the `-o bbbled` install). *Predict* the task and the message.

> **Expect** `do_install` fails with an *invalid user* error from `install`. pseudo checks names against the recipe sysroot's user database, and only useradd.bbclass puts `bbbled` there.

A09-B2 **Drop privileges too early.** Move the `setuid` before opening the LED files, rebuild and deploy. *Predict* what happens on the board.

> **Expect** permission denied on the LED sysfs files (they are root-writable). The daemon exits non-zero. This is why the order in the specification matters.

> **Done when** H1, H2 and T1–T3 pass on a sysvinit image, and B2's fix is back in place.

PHASE 3 Integrating with the platform

A10 · Changing recipes you don’t own: bbappends

You build	a base-files bbappend (hostname, login banner) and a busybox config fragment (`devmem` , `i2cdetect`)
New concept	bbappend matching, `FILESEXTRAPATHS` , the **FILESPATH search order**, config fragments for Kconfig-based recipes
Builds on	A01 (FILESPATH basics)
Activation energy	Medium. Part 1 is designed to fail silently the first time
Ladder link	busybox (cml1, fragments)

Why this exists. You will spend more of your career changing other people’s recipes than writing new ones. Editing `meta/` directly is a fork you would have to maintain forever. A bbappend is a patch to *metadata*, kept in your layer. The catch is that you are now one of several layers competing to supply the same file. Who wins is decided by a rule you have probably never read.

Part 1: base-files (hostname and banner)

Case 1, the obvious way. Create `recipes-core/base-files/base-files_%.bbappend` containing `FILESEXTRAPATHS:prepend := "${THISDIR}/files:"` and `hostname = "bbb-lab"` . Put your banner in `files/motd` . Build and deploy (reflash or install the ipk), then run A10-T1.

You should find that the hostname changed and the banner **did not**. That is A10-B1. Diagnose it before you read on:

```
var base-files FILESOVERRIDES
var base-files FILESPATH | tr ':' '\n' | grep -n -E 'meta-bbb-lab|meta-poky' | head -12
```

`base_set_filespath()` (in `meta/classes-global/utils.bbclass`) loops over **overrides on the outside and directories on the inside**. A file in *any* layer’s override subdirectory therefore beats a plain `files/motd` in every layer. `meta-poky` ships `files/poky/motd` . That file carries the 5.0 “Poky is a reference distro” warning and is selected because `poky` is in `FILESOVERRIDES` .

Case 2, the fix. Pick one approach and justify it in your notebook:

- a. put your file in `files/poky/motd` . This only works while `DISTRO = "poky"` , and the winner between two `poky/` directories then depends on `FILESEXTRAPATHS` order.
- b. ship a uniquely named `bbb-lab-motd` and install it over `/etc/motd` in `do_install:append` . Nothing competes for that name.

Part 2: busybox config fragment

Generate the fragment the supported way:

```
bitbake busybox -c menuconfig      # enable devmem, i2cdetect, i2cget, i2cdump
bitbake busybox -c diffconfig      # writes fragment.cfg into busybox's WORKDIR
```

Copy it into your layer as `recipes-core/busybox/files/bbb-lab.cfg` and add it with a `busybox_%.bbappend` . busybox merges every `*.cfg` in `SRC_URI` with `merge_config.sh` in `do_configure` . All four applets are disabled in poky’s `defconfig` .

What you learn

- `%` in bbappend names, and `bitbake-layers show-appends` .
- `FILESEXTRAPATHS:prepend` (and why `:=` matters inside a bbappend: `THISDIR` must be expanded now, not later).
- The FILESPATH order rule. It also explains why machine-specific files go in `files/<machine>/` .
- `menuconfig` → `diffconfig` → fragment, the same workflow you use for the kernel in A13.

Tests

A10-H1 **Who appends what**

```
bitbake-layers show-appends | grep -A4 -E '^(base-files|busybox)'
```

Expect base-files has two bbappends (meta-poky’s and yours). busybox has yours.

A10-H2 **Variables, not hope**

```
var base-files hostname
grep -E '^CONFIG_(DEVMEM|I2CDETECT|I2CGET|I2CDUMP)=y' $(var busybox B)/.config
```

> **Expect** `bbb-lab`, and four `=y` lines.

A10-T1 **Identity**

```
bbb 'hostname; cat /etc/motd'
```

> **Expect** `bbb-lab` and your banner (this passes only after Case 2).

A10-T2 **Read the SoC's identity register**

```
bbb 'v=$(devmem 0x44E10600 32); echo $v; printf "%X\n" $(( (v >> 12) & 0xFFFF ))'
```

> **Expect** a value such as `0x2B94402E`, and part number `B944` (AM335x). Check the `DEVICE_ID` register (control module + 0x600) in the AM335x TRM (SPRUH73) yourself. Reading the TRM is part of the test.

A10-T3 **Look at the I²C bus**

```
bbb 'i2cdetect -y -r 0'
```

> **Expect** `UU` at 0x24 (the TPS65217 PMIC, claimed by its driver) and a plain `50` at 0x50: the board EEPROM, which **no driver has claimed yet**. Write down what 0x50 shows. In A13 it changes.

Break it (Inversion)

A10-B1 **The silent override.** This is Case 1 above. Record your prediction *before* running A10-T1, then record the diagnosis from the FILESPATH listing.

> **Expect** hostname changed, banner did not. No warning or error anywhere.

A10-B2 **A typo in a fragment.** Change one symbol in `bbb-lab.cfg` to `CONFIG_DEVMEMM=y` and rebuild. *Predict*: will bitbake warn you?

> **Expect** no `WARNING` from bitbake, and `devmem` is gone from the board. Whatever Kconfig said about the unknown symbol (if anything) stayed inside `log.do_configure`. A10-H2 is the check that catches this. That is why you verify `.config` rather than trusting the fragment.

> **Done when** H1, H2 and T1–T3 pass with approach (a) or (b) justified, and B2 is reverted.

A11 · Your image: bbb-lab-image and packagegroup-bbb-lab

You build	`packagegroup-bbb-lab`, `bbb-lab-image`, a rootfs post-process hook, and an image-level license policy
New concept	image recipes in practice, packagegroups, `ROOTFS_POSTPROCESS_COMMAND` + `vardepsexclude`, buildhistory, `INCOMPATIBLE_LICENSE`
Builds on	A01–A10 (everything you have packaged so far)
Activation energy	Medium
Ladder link	your `core-image.bbclass` / `image.bbclass` / `do_rootfs` study, now as the author

> **Why this exists.** So far the image content has lived in `local.conf`, which is a property of *your build directory* rather than of the product. An image recipe makes the content versioned, reviewable and reproducible on another machine. A packagegroup adds one level of naming, so the image says "the lab apps" and does not need editing every time an app is added.

What you write

- `recipes-core/packagegroups/packagegroup-bbb-lab.bb`: `inherit packagegroup`, `LICENSE = "MIT"`, and `RDEPENDS:${PN}` = your apps (`bbb-hello bbbctl bbbinfo bbb-ledd`). Libraries arrive through shlibs (A04). Do not list them.
- `recipes-core/images/bbb-lab-image.bb`: `inherit core-image`, an explicit `IMAGE_INSTALL` (`packagegroup-core-boot packagegroup-bbb-lab ${CORE_IMAGE_EXTRA_INSTALL}`), `IMAGE_FEATURES += "ssh-server-dropbear package-management"`, and `INCOMPATIBLE_LICENSE = "GPL-3.0* LGPL-3.0* AGPL-3.0*"`.
- A shell function in the image recipe, added with `ROOTFS_POSTPROCESS_COMMAND += "bbb_lab_release"` (since 4.3 the separator is a space, not a semicolon), that writes `/etc/bbb-lab-release` containing image name, `DISTRO_NAME`, `DISTRO_VERSION` and build time (`DATETIME`). **Requirement:** re-running `bitbake bbb-lab-image` with nothing changed must not redo `do_rootfs`.
- Clean up `local.conf`: remove every `IMAGE_INSTALL:append` and your `EXTRA_IMAGE_FEATURES` additions (keep `debug-tweaks`). Add `INHERIT += "buildhistory"` and `BUILDHISTORY_COMMIT = "1"`.

What you learn

- Why packagegroups are `allarch`, and what an empty package with `RDEPENDS` really is.
- Hooks that run inside `do_rootfs`, and why time-varying variables need `[vardepsexclude]` (base-files does exactly this with `DATE`).
- buildhistory as the image's audit trail (`image-info.txt`, `installed-package-sizes.txt`, `files-in-image.txt`) and `buildhistory-diff`.
- License policy enforced at image time, and the license manifest.

Tests

A11-H1 What is in the image

```
M=$(ls tmp/deploy/images/beaglebone-yocto/bbb-lab-image-beaglebone-yocto.rootfs.manifest)
grep -E '^(bbb-hello|bbbctl|bbbinfo|bbb-ledd|libbbbhw1|dropbear|opkg) ' $M
```

> **Expect** all seven lines. `libbbbhw1` is there although nobody listed it.

A11-H2 The audit trail

```
ls buildhistory/images/beaglebone_yocto/glibc/bbb-lab-image/
grep IMAGE_FEATURES buildhistory/images/beaglebone_yocto/glibc/bbb-lab-image/image-info.txt
```

> **Expect** `image-info.txt`, `installed-package-sizes.txt`, `files-in-image.txt`, …, and `IMAGE_FEATURES` contains `ssh-server-dropbear package-management`.

A11-H3 No needless rebuilds

Run `bitbake bbb-lab-image` twice, at least one minute apart, with no changes.

> **Expect** the second run executes no image tasks (the task summary says everything was already up to date).

A11-H4 Measuring a change

Add `strace` to the packagegroup, rebuild, and run `buildhistory-diff`. Then remove it again.

> **Expect** the diff names `strace` (plus anything it pulled in) and the image size change.

A11-H5 License manifest

```
find tmp/deploy/licenses -name license.manifest -path '*bbb-lab-image*' | head -1 | xargs grep -A3 'PACKAGE NAME: bbbctl'
```

> **Expect** an entry for `bbbctl` with `LICENSE: MIT`.

A11-T1 **Release file and services**

```
bbb 'cat /etc/bbb-lab-release; hostname; ps | grep -c "[b]bb-ledd"'
```

> **Expect** your release file, `bbb-lab`, and `1`.

Break it (Inversion)

A11-B1 **Time leaks into the hash.** Remove your `[vardepsexclude]` and repeat A11-H3. *Predict*.

> **Expect** `do_rootfs` re-runs on every build a minute apart, because `DATETIME` changes the function's signature. Restore it.

A11-B2 **Policy has teeth.** Add `bash` to `IMAGE_INSTALL`. *Predict* which task fails and why.

> **Expect** the image build refuses `bash` (GPL-3.0-or-later) with an incompatible-license error, at rootfs time rather than when `bash` itself builds. Remove it.

> **Done when** H1–H5 and T1 pass with a clean `local.conf`, and both B tests are reverted.

A12 · Package feeds, versions and on-target upgrades

You build	`libbbbbhw` 0.2 (ABI-compatible addition), `bbbctl` 0.3, a package feed served from your PC
New concept	`package-index`, `PACKAGE_FEED_*`, the PR service, `PV` / `PKGV` / `PKGR`, **version-going-backwards**
Builds on	A11 (image with `package-management`, buildhistory enabled)
Activation energy	Medium
Ladder link	your `do_package_write_ipk` study

> **Why this exists.** A product in the field is updated by packages, not reflashed. That turns version numbers into a contract: a device only accepts *strictly newer* packages. A12 shows how OE keeps that promise (PR service) and how it stops you from breaking it (version QA).

What you build

- `libbbbbhw` 0.2 adds `int bbbhw_led_get(const char *led)` (current brightness or `-errno`). Adding a function is ABI-compatible, so the **SONAME stays `libbbbbhw.so.1`** and the real name becomes `libbbbbhw.so.1.1.0`. Tag `v0.2`.
- `bbbctl` 0.3 adds `bbbctl led <name> get` and `bbbctl led all on|off`. Tag `v0.3`.
- Rename both recipes and bump `SRCREV`.

What you write (local.conf)

```
PRSERV_HOST = "localhost:0"
PACKAGE_FEED_URIS = "http://<your-PC-IP>:8000"
PACKAGE_FEED_BASE_PATHS = "ipk"
```

Rebuild and reflash `bbb-lab-image` **once**, because the feed configuration is written into the image. From then on, deploy with:

```
bitbake package-index
(cd tmp/deploy && python3 -m http.server 8000)      # in a second terminal
```

What you learn

- How the feed configuration ends up in `/etc/opkg/` and which architecture subdirectories it lists.
- `PV` (recipe), `PKGV` (+ git revision), `PR` / `PKGR`, and how the PR service extends `PKGR` so a rebuild with *different output* gets a higher version.
- Why SONAME + package-name stability (A03) is what makes an in-place library upgrade safe.
- **version-going-backwards** is in `ERROR_QA` on scarthgap and relies on buildhistory (A11).

Tests

A12-H1 The feed exists

```
ls tmp/deploy/ipk/*/Packages* | head
```

Expect `Packages` index files per architecture directory.

A12-H2 Versions and ABI

```
oe-pkgdata-util read-value PKGV,PKGR libbbbbhw1 bbbctl
readelf -d $(var libbbbbhw D)/usr/lib/libbbbbhw.so.1.1.0 | grep SONAME
```

Expect PKGVs starting `0.2+git` and `0.3+git`. Record the PKGR format the PR server produces. SONAME is still `libbbbbhw.so.1`.

A12-T1 The board knows the feed

```
bbb 'grep -h -r src /etc/opkg/'
```

Expect one line per architecture, pointing at `http://<your-PC>:8000/ipk/<arch>`.

A12-T2 Upgrade in place

```
bbb 'opkg update && opkg list-upgradable'
bbb 'opkg upgrade && bbbctl --version && bbbctl led all off && bbbctl led '"$LED"' get'
```

Expect `libbbbbhw1` and `bbbctl` are upgradable. After the upgrade: `bbbctl 0.3 (libbbbbhw 0.2)`, all LEDs off, and `get` prints `0`. No reflash.

A12-T3 **Same version, new output**

Change only `bbbctl`'s `do_install` (for example install an extra doc file), then rebuild and `bitbake package-index`.

> **Expect** `opkg list-upgradable` offers `bbbctl` again with a higher **PKGR**, and `PV` is unchanged. Explain who incremented it.

Break it (Inversion)

A12-B1 **Going backwards.** Set `PV = "0.1.9+git"` in the libbbbbhw recipe. *Predict*.

> **Expect** QA **version-going-backwards** fails the build. A feed must never offer an older version under the same name. Restore it.

A12-B2 **Turn the PR server off.** Comment out `PRSERV_HOST` and rebuild `bbbctl`. *Predict* what the package version will be and what QA says.

> **Expect** `PKGR` drops back to the bare `PR`, which is lower than what you published, so version QA fails again. Once you publish a feed, versions only ever go up. Restore `PRSERV_HOST`.

> **Done when** H1, H2 and T1–T3 pass, and you have upgraded the board at least twice without reflashing.

PHASE 4 **Kernel space**

A13 · Kernel configuration fragments

You build	a `linux-yocto` bbappend with a config fragment enabling the LED *pattern* trigger and the AT24 EEPROM driver
New concept	linux-yocto config assembly from fragments, `menuconfig` → `diffconfig`, `kernel_configcheck`
Builds on	A10 (bbappend, fragment workflow)
Activation energy	Low to medium. You know the theory, and you already know why hand-editing `.config` is wrong
Ladder link	linux-yocto, `kernel-yocto.bbclass`

Why this exists. You already know the anti-pattern: `do_kernel_configme` regenerates `.config` from fragments every time, so hand edits vanish. A13 is the *positive* half. A fragment states your intent ("this option must be on") and survives kernel upgrades. `kernel_configcheck` then audits whether that intent actually reached the final `.config`, because Kconfig can silently override you.

Precondition (A13-H0)

```
grep -E 'CONFIG_LEDS_TRIGGER_PATTERN|CONFIG_EEPROM_AT24' $(var linux-yocto B)/.config
```

Expect both are absent or `is not set`. The `beaglebone` BSP fragment in yocto-kernel-cache (6.6) enables `CONFIG_I2C_CHARDEV` but not these two. If either is already set in your tree, choose a different unset option (for example `CONFIG_LEDS_TRIGGER_ACTIVITY`) so that the test still proves something.

What you write

```
bitbake linux-yocto -c kernel_configme -f
bitbake linux-yocto -c menuconfig          # enable LEDS_TRIGGER_PATTERN=y, EEPROM_AT24=m
bitbake linux-yocto -c diffconfig          # fragment.cfg appears in the kernel WORKDIR
```

Copy the fragment to `$LAYER/recipes-kernel/linux/files/bbb-lab.cfg` (keep only your two lines) and add a `linux-yocto_%.bbappend` with `FILESEXTRAPATHS:prepend` and `SRC_URI += "file://bbb-lab.cfg"`.

Before you add anything else, predict: will `at24.ko` be on the board? `beaglebone-yocto.conf` sets `MACHINE_EXTRA_RRECOMMENDS = "kernel-modules"`, but your image is built on `packagegroup-core-boot`, which only pulls the machine's *essential* extras. Find out which packagegroups read that variable (`grep -rl MACHINE_EXTRA_RRECOMMENDS <poky>/meta/recipes-core/packagegroups/`). Then add `kernel-module-at24` to `packagegroup-bbb-lab`, rebuild the image and flash it.

What you learn

- How `kernel-yocto.bbclass` assembles `.config`: kernel-cache `.scc` / `.cfg` for `KMACHINE=beaglebone` and `KBRANCH v6.6/standard/beaglebone` (set for your machine in meta-yocto-bsp's bbappend), then *your* fragments.
- Building a module is not shipping it: `=m` produces a `kernel-module-*` package, and something has to pull that package into the image. `MACHINE_EXTRA_RRECOMMENDS` only helps if your image includes a packagegroup that reads it.
- Kconfig dependencies can veto a fragment, and `kernel_configcheck` is how you find out.

Tests

A13-H1 Intent reached `.config`

```
grep -E 'CONFIG_LEDS_TRIGGER_PATTERN=y|CONFIG_EEPROM_AT24=m' $(var linux-yocto B)/.config
bitbake linux-yocto -c kernel_configcheck -f 2>&1 | grep -iE 'PATTERN|AT24' ; echo "audit-lines=$?"
```

Expect both lines in `.config`, and no audit complaint about your symbols (`audit-lines=1`).

A13-H2 The module is packaged

```
oe-pkgdata-util list-pkgs -p linux-yocto | grep at24
```

Expect a `kernel-module-at24-…` package (the kernel version is part of the name).

A13-T1 **Pattern trigger**

```
# on the board: stop your daemon first, it owns the LED
/etc/init.d/bbb-ledd stop
LED=beaglebone:green:usr3
cat /sys/class/leds/$LED/trigger | tr ' ' '\n' | grep -x pattern
echo pattern > /sys/class/leds/$LED/trigger
echo "255 500 255 0 0 500 0 0" > /sys/class/leds/$LED/pattern      # 1 Hz square wave
```

> **Expect** `pattern` is listed and the LED blinks at 1 Hz with the daemon stopped. (Values clamp to `max_brightness`.)

A13-T2 **The EEPROM now has a driver** (compare with A10-T3)

```
bbb 'i2cdetect -y -r 0 | grep "^50"; lsmod | grep at24; hexdump -C -n 16 /sys/bus/i2c/devices/0-0050/eeprom'
```

> **Expect** `UU` at 0x50 where A10-T3 showed `50`. `at24` is loaded (autoloaded via its modalias). The first bytes are `aa 55 33 ee` followed by `A335BNLT`, the BeagleBone board ID that U-Boot also reads.

Break it (Inversion)

A13-B1 **Misspelled symbol.** Use `CONFIG_LEDS_TRIGER_PATTERN=y` in the fragment. *Predict*: does the build fail?

> **Expect** the build succeeds and the trigger is missing. `kernel_configcheck` reports that the requested value did not reach the final `.config`. Note the exact wording. This is A10-B2 again, except that this time a tool exists to catch it.

A13-B2 **Fight a dependency.** Add `# CONFIG_LEDS_TRIGGERS is not set` to the fragment alongside `PATTERN=y`. *Predict* which one wins.

> **Expect** Kconfig resolves the dependency and drops `PATTERN`, and the audit names the mismatch. (Don't flash this build. Heartbeat and every other trigger would be gone too.) Revert.

> **Done when** H0–H2, T1 and T2 pass.

A14 · An out-of-tree kernel module: bbblab.ko

You build	`bbblab.ko`: a misc device `/dev/bbblab` plus an LED trigger named `bbblab`
New concept	module.bbclass in practice, module packaging, `KERNEL_MODULE_AUTOLOAD`, `KERNEL_MODULE_PROBECONF`
Builds on	A13
Activation energy	Medium. You have read `module.bbclass` line by line; this is the first time you feed it
Ladder link	lttng-modules; your `mydriver.bb` running example

> **Why this exists.** An out-of-tree module must be built against *exactly* the kernel it will load into: same config, same `Module.symvers`, same vermagic. You have already studied how `do_shared_workdir` publishes `STAGING_KERNEL_BUILDDIR` for this purpose. A14 is the consumer side of that contract.

What you build

Create the repository `$SRC/bbblab-mod` with `bbblab.c`, a `Makefile` in the `meta-skeleton/recipes-kernel/hello-mod` style (`KERNEL_SRC`, `M=`, a `modules_install` target), and `COPYING` (GPL-2.0).

- A misc device named `bbblab`. `read()` returns `bbblab: <greeting> #<n>\n`, where `n` increments on every `open()`.
- `module_param(greeting, charp, 0444)`, default `hello`.
- An LED trigger registered as `bbblab` (`led_trigger_register_simple`). Writing `on` or `off` to `/dev/bbblab` calls `led_trigger_event()` with full or zero brightness.

What you write

`bbblab-mod_0.1.bb`: `inherit module`, git `SRC_URI`, `LICENSE = "GPL-2.0-only"`, `RPROVIDES:${PN} += "kernel-module-bbblab"` (meta-skeleton does the same), `KERNEL_MODULE_AUTOLOAD += "bbblab"`, `KERNEL_MODULE_PROBECONF += "bbblab"` and `module_conf_bbblab = "options bbblab greeting=hi"`. Add `kernel-module-bbblab` to your packagegroup.

What you learn

- The full chain: `virtual/kernel:do_shared_workdir` → your `do_configure`/`do_compile` with `KERNEL_SRC=${STAGING_KERNEL_DIR}` and `O=${STAGING_KERNEL_BUILDDIR}` → `modules_install` into `${D}` → `kernel-module-split` packaging.
- Package naming: `kernel-module-bbblab-<kernel version>`, and what the `RPROVIDES` buys you.
- `/etc/modules-load.d/` and `/etc/modprobe.d/` generated from recipe variables.
- Why a kernel config change rebuilds a module whose source did not change.

Tests

A14-H1 Packaging

```
oe-pkgdata-util list-pkgs -p bbblab-mod
oe-pkgdata-util list-pkg-files -p bbblab-mod | grep -E '\.ko|modules-load|modprobe'
```

> **Expect** a `kernel-module-bbblab-<ver>` package containing `bbblab.ko`, `/etc/modules-load.d/bbblab.conf` and `/etc/modprobe.d/bbblab.conf`.

A14-H2 ABI match

```
strings $(find $(var bbblab-mod D) -name bbblab.ko) | grep '^vermagic='
cat $(var linux-yocto STAGING_KERNEL_BUILDDIR)/kernel-abiversion
```

> **Expect** the vermagic release equals `kernel-abiversion`.

A14-H3 The edge you studied

```
bitbake -g bbblab-mod && grep -E '"bbblab-mod\.do_[a-z_]+" -> "linux-yocto\.do_shared_workdir"' task-depends.dot
```

> **Expect** at least one edge into `linux-yocto.do_shared_workdir`.

A14-T1 Loaded at boot, configured by modprobe.d

```
bbb 'lsmod | grep bbblab; cat /dev/bbblab; cat /dev/bbblab; cat /sys/module/bbblab/parameters/greeting'
```

> **Expect** loaded, then `bbblab: hi #1`, `bbblab: hi #2` and `hi`.

A14-T2 **Your trigger drives a real LED**

```
# on the board (bbb-ledd stopped)
echo bbblab > /sys/class/leds/$LED/trigger
echo on  > /dev/bbblab; cat /sys/class/leds/$LED/brightness
echo off > /dev/bbblab; cat /sys/class/leds/$LED/brightness
```

> **Expect** `bbblab` appears in the trigger list. Brightness is max, then `0`, and you can see the LED switch.

Break it (Inversion)

A14-B1 **Move the kernel under the module.** Add one more option to `bbb-lab.cfg` (for example `CONFIG_LEDS_TRIGGER_ACTIVITY=y`) and run `bitbake bbblab-mod`. *Predict*: does the module recompile even though its source did not change?

> **Expect** yes. Its tasks depend on the kernel's published build directory, and a different kernel means a potentially different ABI. Explain what would happen on the board if OE *didn't* rebuild it.

A14-B2 **No autoload.** Remove `KERNEL_MODULE_AUTOLOAD` and reboot a rebuilt image. *Predict*: is `bbblab` loaded?

> **Expect** no. Nothing on this system asks for it. Keep this result: A15 makes it load automatically without putting this variable back.

> **Done when** H1–H3, T1 and T2 pass, and B2's observation is recorded.

A15 · Device tree and a platform driver: binding bbblab to hardware

You build	a kernel patch adding a `bbb-lab` node to `am335x-boneblack.dts`, and `bbblab` 0.2 as a platform driver
New concept	kernel patches in a linux-yocto bbappend, dtb deployment to the boot partition, DT → modalias → udev → modprobe
Builds on	A14
Activation energy	High. Kernel API drift in 6.6, plus patch handling, plus the boot path
Ladder link	linux-yocto, u-boot, and your `kernel-devicetree` study queue

> **Why this exists.** In A14 the module loaded because a *config file* told it to. On real hardware, drivers load because the **hardware description** says the device exists. The device tree creates a platform device, its `modalias` tells udev which module to load, and the module's `of_match_table` binds to it. Configuration then moves out of `modprobe.d` and into the DT, where it belongs.

What you build

The DT node. In `arch/arm/boot/dts/ti/omap/am335x-boneblack.dts` (6.6 moved the TI DTS files under `ti/omap/`, which matches the machine's `KERNEL_DEVICETREE`), add:

```
/ {
    bbb_lab: bbb-lab {
        compatible = "dipesh,bbb-lab";
        bbblab,greeting = "hello from the device tree";
        bbblab,blink-ms = <250>;
    };
};
```

Create the patch from the kernel's own git tree:

```
bitbake linux-yocto -c devshell
#   edit the .dts, then:  git commit -am "ARM: dts: am335x-boneblack: add bbb-lab node"
#                         git format-patch -1 -o $LAYER/recipes-kernel/linux/files/
```

Add `Upstream-Status: Inappropriate [lab exercise]` to the patch header, and add the patch to `SRC_URI` in your linux-yocto bbappend.

The driver (bbblab 0.2). Convert the module to a `platform_driver` named `bbblab` with an `of_match_table` for `dipesh,bbb-lab` and `MODULE_DEVICE_TABLE(of, …)`. In `probe`:

- Read `bbblab,greeting` (falls back to the module parameter) and `bbblab,blink-ms` (default 500).
- Register the misc device and the trigger.
- Expose device attributes `greeting` (read-only) and `blink_ms` (read-write).
- Log `bbblab: probed: greeting="…" blink-ms=…`.

Writing `blink` to `/dev/bbblab` calls `led_trigger_blink()` with `blink_ms` on/off. **API-drift check:** open `include/linux/leds.h` and `include/linux/platform_device.h` in `STAGING_KERNEL_DIR` and use the 6.6 signatures, including the `void`-returning remove callback. Remove `KERNEL_MODULE_AUTOLOAD` and the probeconf from the recipe.

What you learn

- How `kernel-yocto.bbclass` applies bbappend patches to the kernel git tree, and why patch QA applies: since 4.3, patches added by a bbappend to a *core* recipe must carry `Upstream-Status`.
- dtb deployment: `KERNEL_DEVICETREE` → `DEPLOY_DIR_IMAGE` → `IMAGE_BOOT_FILES` / `DTB_FILES` → the vfat boot partition. U-Boot picks `am335x-boneblack.dtb` for this board.
- `oe-run-native`, which runs any native tool (here `dtc`) from the build without installing it on your PC.
- The binding contract: node `compatible` ⇄ `of_match_table` ⇄ `MODULE_DEVICE_TABLE` ⇄ udev modalias.

Tests

A15-H1 **Patch applied cleanly**

```
grep -c 'dipesh,bbb-lab' $(var linux-yocto S)/arch/arm/boot/dts/ti/omap/am335x-boneblack.dts
grep -i fuzz $(var linux-yocto T)/log.do_patch ; echo "fuzz-lines=$?"
```

> **Expect** a count of 1 or more, and `fuzz-lines=1`.

A15-H2 **The node is in the deployed dtb**

```
bitbake dtc-native -c addto_recipe_sysroot
oe-run-native dtc-native dtc -I dtb -O dts tmp/deploy/images/beaglebone-yocto/am335x-boneblack.dtb 2>/dev/null | grep -A4 'bbb-
lab {'
```

> **Expect** the node with your three properties (the `blink-ms` value appears as `<0xfa>`).

A15-H3 **The module advertises the match**

```
strings $(find $(var bbblab-mod D) -name bbblab.ko) | grep '^alias=of:'
```

> **Expect** an `of:N*T*Cdipesh,bbb-lab…` alias.

A15-T1 **The kernel sees the hardware description**

```
bbb 'cat /proc/device-tree/bbb-lab/compatible; echo; cat /sys/bus/platform/devices/bbb-lab/modalias'
```

> **Expect** `dipesh,bbb-lab`, and a modalias beginning `of:Nbbb-lab` and containing `Cdipesh,bbb-lab`.

A15-T2 **Autoloaded with no autoload config** (compare with A14-B2)

```
bbb 'lsmod | grep bbblab; dmesg | grep "bbblab: probed"; ls /sys/bus/platform/drivers/bbblab/; cat
/sys/bus/platform/devices/bbb-lab/blink_ms; cat /dev/bbblab'
```

> **Expect** loaded and probed with the DT values. A `bbb-lab` link under the driver. `250`. `bbblab: hello from the device tree #1`.

A15-T3 **Unbind and bind** (tests your remove path)

```
bbb 'echo bbb-lab > /sys/bus/platform/drivers/bbblab/unbind; ls /dev/bbblab; echo bbb-lab >
/sys/bus/platform/drivers/bbblab/bind; ls /dev/bbblab'
```

> **Expect** `No such file or directory` after unbind, the device back after bind, and no warnings in `dmesg`.

Break it (Inversion)

A15-B1 **Patch without Upstream-Status.** Remove the line and rebuild. *Predict* the QA outcome.

> **Expect** a **patch-status** QA complaint. The check follows the layer of the *recipe* (linux-yocto lives in core), not the layer of your bbappend. Restore the line.

A15-B2 **Break the contract.** Change the DT `compatible` to `dipesh,bbblab`, reflash, boot. *Predict*: is the module loaded? Does `probe` run if you `modprobe` it by hand?

> **Expect** not autoloaded (the modalias matches nothing). A manual `modprobe` loads it, but `probe` never runs and the driver directory has no device. Revert.

> **Done when** H1–H3 and T1–T3 pass, and you can draw the DT → modalias → udev → modprobe → probe chain from memory.

A16 · devtool: the edit, build, deploy loop

You build	a new `bbbctl` subcommand, developed in a devtool workspace and landed in two different ways
New concept	the workspace layer, `externalsrc`, `deploy-target` / `undeploy-target`, `devtool finish`, patch vs SRCREV
Builds on	A02–A15 (you now know what devtool automates)
Activation energy	Medium. The tool is simple; knowing which state it leaves behind is not
Ladder link	none. This is how you will touch any recipe from here on

Why this exists. Commit, bump `SRCREV`, rebuild, copy the ipk, reinstall: you have done this loop a dozen times by now. devtool shortens it by pointing the recipe at a source tree you edit directly (`externalsrc`) and copying `${D}` straight onto the board. You have to learn devtool *after* the manual loop, because devtool leaves state behind. If you don't know what it changed, you can't clean up after it.

What you build

`bbbctl led <name> trigger [TRIGGER]`: without an argument, print the *active* trigger (the bracketed entry in sysfs). With one, set it.

```
devtool modify bbbctl                    # creates workspace/, checks out the source
# edit workspace/sources/bbbctl, then:
devtool build bbbctl
devtool deploy-target bbbctl root@$BBB    # copies ${D} to the board, backing up what it overwrites
#  ... test, iterate ...
git -C workspace/sources/bbbctl commit -am "bbbctl: add 'led <name> trigger'"
devtool finish bbbctl $LAYER              # turns your commit into a patch in your layer
```

Then **land it properly**. You own upstream, so a patch in your layer is the wrong final form. Apply the commit to `$SRC/bbbctl` (`git am` the generated patch), tag `v0.4`, bump `SRCREV`, rename the recipe, and delete the patch. Write down when a patch in the layer *is* the right form.

What you learn

- What `devtool modify` creates: the `workspace` layer, `workspace/appends/bbbctl_*.bbappend` and `EXTERNALSRC`.
- `externalsrc` removes `do_fetch` / `do_unpack` / `do_patch` and builds from your tree.
- `deploy-target` bypasses packaging: no `postinst`, no dependency resolution. Know what you are skipping.
- `devtool finish` writes patches. `devtool reset` throws your work away. Choose deliberately.

Tests

A16-H1 **Workspace in effect** (while modifying)

```
devtool status
bitbake-layers show-layers | grep workspace
var bbbctl EXTERNALSRC
bitbake -c listtasks bbbctl | grep -cE 'do_(fetch|unpack|patch)\b'
```

Expect `bbbctl` listed with its source path, the workspace layer present, `EXTERNALSRC` pointing at `workspace/sources/bbbctl`, and a count of `0` (those tasks are gone).

A16-T1 **Deployed without packaging**

```
bbb "bbbctl led $LED trigger; bbbctl led $LED trigger heartbeat; bbbctl led $LED trigger"
```

Expect the current trigger, then `heartbeat`, and the LED pulsing.

A16-T2 **Undeploy restores**

```
devtool undeploy-target bbbctl root@$BBB
bbb "bbbctl led $LED trigger"; bbb 'echo rc=$?'
```

Expect the packaged `bbbctl` is back, and the new subcommand is rejected.

A16-H2 **After finish, and after landing**

```
git -C $LAYER status --short      # right after devtool finish
devtool status
```

> **Expect** a new `.patch` plus a modified recipe, and `bbbctl` gone from the workspace. After you land it upstream, `git status` shows only the renamed recipe with a new `SRCREV`.

Break it (Inversion)

A16-B1 **The forgotten workspace.** Run `devtool modify bbbctl` again. Without finishing, edit the *layer* recipe (for example set `SRCREV` to the `v0.1` commit) and run `bitbake bbbctl`. *Predict* which source gets built.

> **Expect** the workspace source. `bitbake-layers show-appends | grep -A2 bbbctl` shows the workspace bbappend overriding your layer. This trap catches people weeks later. Clean up with `devtool reset bbbctl` and restore the `SRCREV`.

> **Done when** H1, H2, T1 and T2 pass, and `bbbctl` 0.4 is landed through `SRCREV` with no patch left in the layer.

A17 · The SDK: developing outside BitBake

You build	`ledbar`, a 4-LED binary counter, developed with the SDK, then turned into a recipe with `devtool add`
New concept	`populate_sdk`, the SDK sysroots and `environment-setup`, SDK manifests, `devtool add`/recipetool
Builds on	A11 (image), A16 (devtool)
Activation energy	Medium
Ladder link	none (your `populate_sdk.bbclass` study is deferred; this gives it a reason)

Why this exists. Application developers on a team do not run BitBake. They get an SDK that *matches the image*: same compiler, same tune flags, same library versions. An SDK is a snapshot of one image's build-time world, and like every snapshot it goes stale (B2).

What you build

1. Build and install the SDK:

```
bitbake bbb-lab-image -c populate_sdk
ls tmp/deploy/sdk/            # poky-glibc-x86_64-bbb-lab-image-cortexa8hf-neon-beaglebone-yocto-toolchain-*.sh
tmp/deploy/sdk/poky-glibc-x86_64-bbb-lab-image-*-toolchain-*.sh -d ~/sdk/bbb-lab -y
```

2. In a **new** terminal (never in the bitbake shell), run `source ~/sdk/bbb-lab/environment-setup-cortexa8hf-neon-poky-linux-gnueabi`.
3. Create the repository `$SRC/ledbar` (C + Makefile, `$(CC)` and `pkg-config bbbhw`). It saves each LED's current trigger, counts 0–15 in binary across the four user LEDs at 250 ms per step, then restores the saved triggers.
4. Build it with plain `make` in the SDK shell, `scp` it to the board and run it.
5. Productize it: `devtool add --srcrev <commit> bbb-ledbar "git://${BBB_LAB_SRC}/ledbar;protocol=file;branch=main"`. Read what recipetool generated, fix what it could not infer, `devtool finish`, and add it to the packagegroup.

What you learn

- What the SDK installer contains: a native sysroot (cross gcc, pkg-config, ...) and a target sysroot (headers and libs of everything in the image, because `SDKIMAGE_FEATURES` adds the `-dev` packages).
- What `environment-setup-*` exports (`CC`, `CFLAGS`, `LDFLAGS`, `PKG_CONFIG_SYSROOT_DIR`, `OECORE_TARGET_SYSROOT`), and why those match what your recipes saw.
- What recipetool can infer (fetch, license, build system, install target) and what it cannot (your pkg-config dependency).

Tests

A17-H1 **The SDK knows your library**

```
grep -E '^libbbbhw-dev ' tmp/deploy/sdk/*bbb-lab-image*.target.manifest
```

Expect one line.

A17-H2 **The SDK environment** (in the SDK shell)

```
echo $CC | tr ' ' '\n' | grep -E 'gcc|sysroot'; pkg-config --libs bbbhw; echo $PKG_CONFIG_SYSROOT_DIR
```

Expect the cross gcc with `--sysroot=…/sysroots/cortexa8hf-neon-poky-linux-gnueabi`, `-lbbbhw`, and the same sysroot path.

A17-H3 **What you built**

```
file ledbar; readelf -d ledbar | grep NEEDED
```

Expect ARM hard-float, needing `libbbbhw.so.1` and `libc.so.6`.

A17-H4 **recipetool's guess vs your knowledge**

```
cat workspace/recipes/bbb-ledbar/*.bb      # before you edit it
```

Expect `SRC_URI`, `SRCREV`, a license guess and Makefile-based tasks. Record what was missing and why the first `devtool build` failed until you fixed it.

A17-T1 **Runs and cleans up**

```
bbb 'for l in /sys/class/leds/*; do cat $l/trigger | grep -o "\[[^]]*\]"; done'   # before
scp ledbar root@$BBB:/tmp/ && bbb /tmp/ledbar
bbb 'for l in /sys/class/leds/*; do cat $l/trigger | grep -o "\[[^]]*\]"; done'   # after
```

> **Expect** a visible 0–15 binary count, and identical trigger lists before and after.

A17-T2 **The recipe-built version**

```
bbb 'opkg update && opkg install bbb-ledbar && ledbar'
```

> **Expect** the same behaviour, now from the feed.

Break it (Inversion)

A17-B1 **Wrong compiler.** In the SDK shell, build with `gcc ledbar.c -o ledbar-host $(pkg-config --cflags --libs bbbhw)`, copy it over and run it. *Predict* the board's response.

> **Expect** the kernel refuses an x86-64 ELF (`ENOEXEC`). busybox ash may report a confusing syntax error or `not found` instead of saying so. `file` explains it in one line.

A17-B2 **A stale SDK.** Add `int bbbhw_led_count(void)` to libbbbhw (0.3, compatible, SONAME unchanged), publish it to the feed, use it in `ledbar`, and build in your *existing* SDK. *Predict*.

> **Expect** an implicit-declaration or undefined-reference error, because the SDK's sysroot is a snapshot of the image when it was built. Regenerate and reinstall the SDK, then rebuild. Entropy: the SDK has to be rebuilt whenever the libraries in the image change.

> **Done when** H1–H4, T1 and T2 pass, and `bbb-ledbar` is in your packagegroup.

PHASE 6 **Product: policy, storage, verification**

A18 · Your own distro and machine

You build	`conf/distro/bbblab.conf` (systemd, trimmed features) and `conf/machine/bbb-lab.conf` (derived from `beaglebone-yocto`)
New concept	the three-way split *policy* (DISTRO) / *hardware* (MACHINE) / *content* (image), `MACHINEOVERRIDES`, `INIT_MANAGER`
Builds on	A09 (systemd half of bbb-ledd), A11 (image)
Activation energy	High. Switching the init manager rebuilds most of the world. Start it before you go to sleep
Ladder link	u-boot (machine-conf variables such as `UBOOT_MACHINE`, `SPL_BINARY`)

Why this exists. Poky is a *reference* distro. Since 5.0 it even adds a warning to `/etc/motd` saying so. A product owns its policy (init system, features, naming) in a distro file, and its hardware description in a machine file. Neither belongs in `local.conf`, which is per build directory, nor in the image, which is content. A18 also shows why BSP layers key so much on overrides: your new machine must *still look like* `beaglebone-yocto` to every recipe that already supports that board.

What you write

`conf/distro/bbblab.conf`: `require conf/distro/poky.conf`, then set `DISTRO = "bbblab"`, `DISTRO_NAME = "BBB Lab Linux"`, `DISTRO_VERSION = "1.0"`, `INIT_MANAGER = "systemd"` and `DISTRO_FEATURES:remove = "x11 wayland vulkan opengl"`. **Keep `ptest`**, because A20 needs it.

`conf/machine/bbb-lab.conf`: `MACHINEOVERRIDES =. "beaglebone-yocto:"` and `require conf/machine/beaglebone-yocto.conf`, plus one real customisation (for example `SERIAL_CONSOLES = "115200;ttyS0"`, dropping the console names the board does not have).

Switch `local.conf` to `DISTRO = "bbblab"` and `MACHINE = "bbb-lab"`, rebuild `bbb-lab-image`, and flash it from `tmp/deploy/images/bbb-lab/`.

What you learn

- `INIT_MANAGER` and what `init-manager-systemd.inc` turns on (`systemd`, `usrmerge`, the `VIRTUAL-RUNTIME_*` choices).
- `MACHINEOVERRIDES`: why `linux-yocto` (via meta-yocto-bsp's `COMPATIBLE_MACHINE:beaglebone-yocto`, `KMACHINE:beaglebone-yocto`, `KBRANCH:beaglebone-yocto`) still accepts your machine.
- `DISTRO_FEATURES` vs `MACHINE_FEATURES` vs `COMBINED_FEATURES`.
- sstate partitioning: tune-specific packages (`cortexa8hf-neon`) are shared between machines with the same tune. Machine-specific ones are not.
- Second-order check: your A10 banner. If you chose approach (a) (`files/poky/motd`), predict what happens now that `poky` is no longer in `FILESOVERRIDES`.

Tests

A18-H1 **Policy and hardware, as BitBake sees them**

```
bitbake-getvar -q --value MACHINEOVERRIDES
bitbake-getvar -q --value DISTRO_FEATURES | tr ' ' '\n' | grep -E '^(systemd|usrmerge|ptest|x11|wayland)$'
```

Expect `MACHINEOVERRIDES` contains both `beaglebone-yocto` and `bbb-lab`. The features list `systemd`, `usrmerge` and `ptest`, and neither `x11` nor `wayland`.

A18-H2 **The kernel still knows the board**

```
var linux-yocto KMACHINE; var linux-yocto KBRANCH
```

Expect `beaglebone` and `v6.6/standard/beaglebone`.

A18-H3 **A09's systemd half switched itself on**

```
var bbb-ledd PACKAGECONFIG
oe-pkgdata-util list-pkg-files bbb-ledd | grep -E 'systemd|init\.d'
```

Expect `PACKAGECONFIG` contains `systemd` without any recipe change, because of the DISTRO_FEATURES filter from A08/A09. The unit file is shipped. Check whether the sysvinit script still is, and explain why.

A18-T1 **Identity**

```
bbb 'cat /etc/os-release | grep -E "^(NAME|VERSION_ID)="; cat /etc/motd; hostname'
```

> **Expect** `NAME="BBB Lab Linux"`, `VERSION_ID="1.0"`, no Poky reference-distro warning, your banner (if your A10 approach survived), and `bbb-lab`.

A18-T2 **systemd runtime of bbb-ledd** (deferred from A09)

```
bbb 'systemctl is-active bbb-ledd; systemctl show -p Type bbb-ledd; journalctl -u bbb-ledd -b --no-pager | tail -3; ps | grep "
[b]bb-ledd"'
```

> **Expect** `active`, `Type=notify` (your `sd_notify` works), startup log lines, and the process running as `bbbled`.

Break it (Inversion)

A18-B1 **Remove** `MACHINEOVERRIDES`. *Predict* the first failure.

> **Expect** `linux-yocto` is skipped as *incompatible with machine bbb-lab* (not in `COMPATIBLE_MACHINE`), so nothing provides `virtual/kernel`. The overrides line is what makes your machine "a beaglebone" to existing BSP metadata. Restore it.

A18-B2 **Predict the bill.** Before switching `MACHINE` back to `beaglebone-yocto` (keeping `DISTRO = "bbblab"`), write down which recipes you expect to rebuild. Then run `bitbake bbb-lab-image` and read the `Sstate summary` line and the executed tasks.

> **Expect** high sstate reuse. Machine-arch recipes (kernel, U-Boot, base-files, the image) rebuild. Your tune-arch apps are reused. Switch back to `bbb-lab` when you are done.

> **Done when** H1–H3, T1 and T2 pass on the `bbb-lab` machine with the `bbblab` distro.

A19 · Storage layout: custom WIC, read-only rootfs, persistent /data

You build	`wic/bbb-lab.wks` with a third partition, a read-only root filesystem, and `bbb-ledd` 0.4 keeping state on `/data`
New concept	wic and `.wks` syntax, `read-only-rootfs` and its consequences, tmpfiles.d, mount ordering
Builds on	A18 (systemd, your machine conf)
Activation energy	High
Ladder link	your `do_image_wic` / `image_types` study queue

> **Why this exists.** A writable rootfs on a device that loses power is a corruption risk, and a device that can modify `/usr` is harder to secure and to update. A read-only rootfs fixes both. It also **forces you to declare** everything that needs to persist, and pushes all first-boot configuration back to build time. The second point is why `useradd` (A09) runs at rootfs time and not on first boot.

What you build

- **`$LAYER/wic/bbb-lab.wks`**: start from the machine's `beaglebone-yocto.wks` (vfat `/boot` via `bootimg-partition` with `loader=u-boot`, ext4 `/`, `bootloader --append="console=ttyS0,115200"`) and add `part /data --ondisk mmcblk0 --fstype=ext4 --label data --size 64M --align 4`. Set `WKS_FILE = "bbb-lab.wks"` in `bbb-lab.conf`.
- In the image: `IMAGE_FEATURES += "read-only-rootfs"`.
- **`bbb-ledd` 0.4**: on start, read `boot_count` from the state directory, increment it, write it back, and include `boot_count=N` in `state`. The default state directory becomes `/data/bbb-ledd` (via `/etc/default/bbb-ledd`). The unit gains `RequiresMountsFor=/data`. The recipe ships a tmpfiles.d entry, `d /data/bbb-ledd 0750 bbbled bbbled -`, in `${nonarch_libdir}/tmpfiles.d/`.

What you learn

- wic source plugins (`bootimg-partition`, `rootfs`), empty partitions, and how wic adds `/etc/fstab` entries for partitions with a mount point.
- `wic ls` to inspect an image on the host before flashing it.
- What `read-only-rootfs` changes: no on-target postinsts, and every writable path must be declared (volatile, tmpfs or a real partition).
- Why `tmpfiles.d` is the right tool to create directories on a read-only system.

Tests

A19-H1 Three partitions, inspected before flashing

```
bitbake wic-tools
wic ls tmp/deploy/images/bbb-lab/bbb-lab-image-bbb-lab.rootfs.wic
```

Expect three partitions: fat (boot), ext4 (root), ext4 (data, about 64 MiB).

A19-H2 The feature and the tmpfiles entry

```
var bbb-lab-image IMAGE_FEATURES | tr ' ' '\n' | grep read-only-rootfs
oe-pkgdata-util list-pkg-files bbb-ledd | grep tmpfiles
```

Expect both present.

A19-T1 Read-only root, writable data

```
bbb 'mount | grep -E " on / | on /data "; touch /etc/x; grep /data /etc/fstab'
```

Expect `/` mounted `ro`, `/data` mounted `rw` (ext4), `touch` fails with *Read-only file system*, and an fstab line for `/data` that wic wrote.

A19-T2 State survives reboot

```
bbb 'cat /data/bbb-ledd/state'; bbb reboot; sleep 60; bbb 'cat /data/bbb-ledd/state'
```

Expect `boot_count` increased by exactly 1, and the files are owned by `bbbled`.

A19-T3 Nothing failed

```
bbb 'systemctl --failed --no-legend | wc -l; systemctl is-active bbb-ledd'
```

Expect `0` and `active`. ssh working at all also shows that dropbear coped with a read-only `/etc`.

Break it (Inversion)

A19-B1 **A first-boot script.** Write a tiny recipe `bbb-firstboot` with `pkg_postinst_ontarget:${PN}() { echo done > /etc/bbb-firstboot; }`, add it to the image, and build. *Predict* when and how it fails.

> **Expect** `do_rootfs` fails, saying the package could not be configured offline and the rootfs is read-only. This is the Chesterton's fence behind A09's build-time `useradd`. Remove the recipe from the image.

A19-B2 **State in the wrong place.** Point `bbb-ledd` back at `/var/lib/bbb-ledd` and reboot twice. *Predict*.

> **Expect** either the daemon fails writing its state (read-only) or `boot_count` resets every boot (a volatile `/var/lib`). Find out which applies to your image and explain why. Either outcome shows that persistence has to be designed. Revert.

Done when H1, H2 and T1–T3 pass, and both B tests are explained.

A20 · Capstone: test automation with ptest and testimage

You build	a ptest suite for `libbbbhw`, a test image variant, and an **oeqa runtime suite** that re-runs the T-tests of A01–A19 against the real board
New concept	ptest.bbclass, `ptest-runner`, oeqa runtime test cases, `testimage` with `TEST_TARGET = "simpleremote"`, image variants with `require`
Builds on	everything
Activation energy	High at the start, low afterwards. Once it runs, every future change is checked in one command
Ladder link	busybox (it inherits ptest)

> **Why this exists.** You have run more than fifty target tests by hand. Manual tests decay: you stop re-running the old ones, and then a change in A19 quietly breaks A09. The Yocto Project's own answer is two layers of automation. **ptest** packages a component's tests and runs them on the device. **oeqa/testimage** drives a running device from the build host and asserts on the results. A20 turns your notebook into a regression suite.

What you build

ptest for libbbbhw. Add `tests/test-bbbhw.c` to the library repository. It prints one automake-style line per check (`PASS: name` / `FAIL: name`) and exits non-zero on any failure. Cover:

- `bbbhw_parse_pattern()`: one valid input and three invalid ones.
- `bbbhw_version()` matches `BBBHW_VERSION`.
- `bbbhw_board_model()` returns the BBB model (a hardware check).

Add a `check-build` Makefile target that builds the test binary. In the recipe: `inherit ptest`, a `run-ptest` script in `SRC_URI`, `do_compile_ptest` and `do_install_ptest` into `${PTEST_PATH}`.

A test image. `bbb-lab-image-test.bb`: `require bbb-lab-image.bb` and `IMAGE_INSTALL += "ptest-runner libbbbhw-ptest"`. Flash it. `simpleremote` tests whatever is *already running*, it does not deploy.

The oeqa suite. Create `$LAYER/lib/oeqa/runtime/cases/bbblab.py` with one test method per assignment checkpoint. A starting point:

```
from oeqa.runtime.case import OERuntimeTestCase
from oeqa.core.decorator.depends import OETestDepends

class BbbLabTest(OERuntimeTestCase):

    @OETestDepends(['ssh.SSHTest.test_ssh'])
    def test_a01_hello(self):
        status, output = self.target.run('bbb-hello')
        self.assertEqual(status, 0, msg=output)
        self.assertIn('board: TI AM335x BeagleBone Black', output)

    # test_a04_bbbctl_version, test_a08_bbbinfo_json, test_a09_ledd_unprivileged,
    # test_a10_identity, test_a12_feed_configured, test_a13_eeprom_bound,
    # test_a15_dt_binding, test_a18_systemd_notify, test_a19_ro_root_data_rw,
    # test_a20_libbbbhw_ptest  (runs: ptest-runner libbbbhw)  ... you write these
```

In `local.conf`:

```
IMAGE_CLASSES += "testimage"
TEST_TARGET = "simpleremote"
TEST_TARGET_IP = "<board IP>"
TEST_SERVER_IP = "<your PC IP>"
TEST_SUITES = "ping ssh bbblab"
```

Run `bitbake bbb-lab-image-test -c testimage`.

What you learn

- ptest packaging (`${PN}-ptest`, `PTEST_PATH`, `run-ptest`) and the output format `ptest-runner` collects.
- oeqa: `self.target.run()`, assertion messages that explain themselves, and `OETestDepends` (with `OEHasPackage` for package-conditional tests).
- `testimage` against real hardware, where results are written, and why `simpleremote` assumes you already deployed.
- Image variants with `require`, so test-only packages never reach the product image.

Tests

A20-H1 **The ptest package**

```
oe-pkgdata-util list-pkg-files libbbbhw-ptest
```

> **Expect** `run-ptest` and `test-bbbhw` under `/usr/lib/libbbbhw/ptest/`.

A20-T1 **ptest on the board**

```
bbb 'ptest-runner libbbbhw; echo rc=$?'
```

> **Expect** every line `PASS:` and `rc=0`.

A20-H2 **The whole suite, one command**

```
bitbake bbb-lab-image-test -c testimage
find tmp/log/oeqa -name 'testresults.json' | head -1
```

> **Expect** `ping`, `ssh` and all your `bbblab` tests pass, and a JSON result file exists. Run `resulttool report` on it for a summary.

Break it (Inversion)

A20-B1 **Do the tests have teeth? (mutation test).** With `devtool modify libbbbhw`, make `bbbhw_parse_pattern()` accept a single number. `devtool deploy-target` it and run testimage again. *Predict* exactly which test fails and what its message says.

> **Expect** the ptest check for that invalid input fails, and so does your `test_a20_libbbbhw_ptest`, with a message that names the case. If nothing failed, your tests don't cover what you think they cover. Fix the tests, not the mutation. Then `devtool undeploy-target` and `devtool reset`.

A20-B2 **Board unplugged.** Power the BBB off and run testimage. *Predict* the result of every test.

> **Expect** `ping` **fails**. `ssh` and everything that depends on it is **skipped**, not failed. That is `OETestDepends` at work. Make sure every one of your tests declares the dependency.

Done when H1, H2 and T1 pass, and the B1 mutation is caught by name. From now on, `bitbake bbb-lab-image-test -c testimage` is your definition of "the system works".

Mastery roadmap

The 20% that carries 80% (Pareto)

If you only re-do five assignments a year from now, re-do these. Together they contain the mechanisms that every other assignment reuses:

Assignment	Why it is load-bearing
A04 Consuming the library	per-recipe sysroots and shlibs. Most “it builds on my machine” bugs are here
A07 Native tools	target vs native vs host. The second most common class of bugs
A10 bbappends	FILESPATH order and silent overrides. You will meet this every week in product work
A15 Device tree + driver	the full hardware → kernel → userspace binding chain on your own board
A20 Test automation	it keeps all the others true

Compound interest: combinations that unlock more than their parts

- **A03 + A12**: SONAME discipline plus monotonic versions = safe in-field library upgrades.
- **A09 + A19**: build-time users plus a read-only rootfs = a system that needs nothing at first boot.
- **A14 + A15 + A13**: module, DT binding and config fragment = a complete board-support change, which is what a BSP layer is made of.
- **A16 + A17**: devtool for platform work plus SDK for application work, the split real teams use.

Inversion checklist: how to guarantee a broken product

Each line is something a previous assignment made you do on purpose. Never do it by accident:

1. Call the cross compiler directly instead of `${CC}` (A01-B1).
2. Accept an upstream `CC = gcc` (A02-B1).
3. Leave `SRCREV = "${AUTOREV}"` in a release (A02-B2).
4. Build a shared library without a SONAME (A04-B2).
5. Rely on something being “already built” instead of declaring it in `DEPENDS` (A04-B1, A08-B2).
6. Depend on a target recipe when you need to run its tool on the build host (A07-B1).
7. Assume a bbappend file override took effect without checking FILESPATH (A10-B1).
8. Trust a config fragment without checking `.config` or running `kernel_configcheck` (A10-B2, A13-B1).
9. Put a timestamp into a task without `vardepsexclude` (A11-B1).
10. Let a published package version go backwards, or turn off the PR service after publishing (A12-B1, A12-B2).
11. Forget a devtool workspace (A16-B1).
12. Hand out an SDK and never rebuild it (A17-B2).
13. Depend on first-boot scripts on a read-only system (A19-B1).
14. Write tests that a deliberate bug doesn’t make fail (A20-B1).

Next steps

- **To ~90%:** write a BSP layer for a second board you own (your Zynq experience transfers here), with its own machine conf, a U-Boot bbappend and a kernel recipe. Then read `populate_sdk_ext` (eSDK) and `image_types.bbclass` at function-signature level. Both are on your study queue.
- **Toward expert:** multiconfig builds (A18 with two machines in one build), `kernel-fitimage` with signed images, an update system (SWUpdate/RAUC) on top of A19’s layout, SPDX/CVE reporting (`create-spdx` is already enabled by poky), and upstreaming a real fix to OE-core. That last one also feeds your kernel-maintainership goal.

Entropy: keeping this knowledge alive

The recipe *syntax* is learn-once. What decays is **version-specific behaviour**: scarthgap’s `S = "${WORKDIR}"` becomes `UNPACKDIR` in later releases, QA lists change, kernel APIs drift (A15). Re-run A20’s testimage after every layer update. Every time you move to a new Yocto release, read its migration guide and redo **A01 and A10** on it.

Appendix A · Command reference

Question	Command
Value of a variable in a recipe	`bitbake-getvar -r <recipe> --value VAR`
Why it has that value	`bitbake-getvar -r <recipe> VAR` (full history)
Same, the classic way	`bitbake -e <recipe> | grep -E '^VAR='`
Which layers and priorities	`bitbake-layers show-layers`
Which bbappends apply	`bitbake-layers show-appends`
All versions of a recipe	`bitbake-layers show-recipes <recipe>`
Packages a recipe produced	`oe-pkgdata-util list-pkgs -p <recipe>` (`-r` for runtime names)
Files in a package	`oe-pkgdata-util list-pkg-files <pkg>` / `-p <recipe>`
Recipe-space ⇄ runtime name	`oe-pkgdata-util lookup-pkg <pkg>` (`-r` reverses)
Who ships a file	`oe-pkgdata-util find-path '/usr/bin/foo'`
Package metadata	`oe-pkgdata-util read-value RDEPENDS,PKGV <runtime-pkg>`
Task dependency graph	`bitbake -g <target>` → `task-depends.dot`
Why a task re-ran	`bitbake-diffsigs -t <recipe> <task>`
Shell inside the build environment	`bitbake <recipe> -c devshell`
Run a native tool	`bitbake <tool>-native -c addto_recipe_sysroot && oe-run-native <tool>-native <cmd>`
Kernel / busybox config	`-c menuconfig`, `-c diffconfig`, `-c kernel_configcheck -f`
Image changes over time	`buildhistory-diff`
Inspect a wic image	`bitbake wic-tools && wic ls <image>.wic`
devtool lifecycle	`modify` → `build` → `deploy-target` → `finish` | `reset`; `add`; `status`

Appendix B · Where things live (scarthgap, beaglebone-yocto)

What	Path (relative to the build directory, default `TMPDIR`)
Target recipe work dir	`tmp/work/cortexa8hf-neon-poky-linux-gnueabi/<pn>/<pv>/` (no PR since 4.3)
Machine-specific recipes	`tmp/work/beaglebone_yocto-poky-linux-gnueabi/<pn>/<pv>/`
Native recipes	`tmp/work/x86_64-linux/<pn>-native/<pv>/`
`S`, `B`, `D`	`WORKDIR/<src or git>`, `WORKDIR/build` (autotools/cmake) or `= S`, `WORKDIR/image`
Recipe sysroots	`WORKDIR/recipe-sysroot/`, `WORKDIR/recipe-sysroot-native/`
Staged per-recipe output	`tmp/sysroots-components/<arch>/<pn>/`
Packages	`tmp/deploy/ipk/{cortexa8hf-neon,beaglebone_yocto,all}/`
Package metadata	`tmp/pkgdata/beaglebone-yocto/`
Kernel shared dirs	`tmp/work-shared/beaglebone-yocto/kernel-source`, `.../kernel-build-artifacts`
Images, dtbs, MLO, u-boot	`tmp/deploy/images/beaglebone-yocto/` (`…/bbb-lab/` after A18)
Licenses	`tmp/deploy/licenses/`
SDK installers	`tmp/deploy/sdk/`
buildhistory	`buildhistory/{packages,images}/…`
devtool	`workspace/{sources,recipes,appends}/`

Appendix C · Concept index

Concept	Where
layer.conf, LAYERSERIES_COMPAT, priority	A01
FILESPATH, FILESEXTRAPATHS, FILESOVERRIDES	A01, **A10**
`S`, `B`, `D`, `WORKDIR`	A01, A05, A06
Licensing, LIC_FILES_CHKSUM, license manifest, INCOMPATIBLE_LICENSE	A01, A02, A11
git fetcher, SRCREV, AUTOREV, PV/PKGV	A02, A12
EXTRA_OEMAKE, oe_runmake, DESTDIR conventions	A02
PACKAGES/FILES order, `-dev` / `-dbg` / `-doc` split	A01, A02, **A03**
debian renaming, recipe-space vs runtime names	A03
DEPENDS, recipe sysroots, do_prepare_recipe_sysroot	**A04**
shlibs and automatic RDEPENDS	A04, A08
PREFERRED_VERSION	A04
autotools.bbclass, EXTRA_OECONF, configure QA	A05
cmake.bbclass, toolchain.cmake, EXTRA_OECMAKE	A06
native / BBCLASSEXTEND / SYSROOT_DIRS	**A07**
PACKAGECONFIG, `??=`, DISTRO_FEATURES filter	A08, A09
Task signatures, bitbake-diffsigs, hash equivalence	A07, A08, A11
update-rc.d, systemd, INIT_MANAGER	A09, A18
useradd, pseudo ownership	A09, A19
CONFFILES	A09
bbappend, `%` wildcard, show-appends	A10, A13, A15
Kconfig fragments (busybox and kernel)	A10, A13
Image recipes, packagegroups, rootfs hooks, vardepsexclude	A11
buildhistory	A11, A12
Package feeds, PR service, version-going-backwards	A12
kernel-yocto config assembly, kernel_configcheck	A13
module.bbclass, autoload, probeconf, vermagic	A14
Kernel patches, Upstream-Status QA, dtb deploy	A15
DT → modalias → udev → probe	A15
devtool modify/build/deploy/finish/reset/add	A16, A17
SDK, environment-setup, manifests	A17
DISTRO vs MACHINE, MACHINEOVERRIDES, sstate reuse	A18
wic/wks, read-only-rootfs, tmpfiles.d	A19
ptest, oeqa, testimage, image variants	A20

Appendix D · Verification notes

Checked against the scarthgap sources (git.yoctoproject.org, `h=scarthgap`) while writing this curriculum:

- `beaglebone-yocto.conf`: tune `cortexa8hf-neon`, `IMAGE_FSTYPES += "… wic wic.bmap"`, `WKS_FILE`, `KERNEL_DEVICETREE` under `ti/omap/`, `DTB_FILES`, `IMAGE_BOOT_FILES`, `MACHINE_EXTRA_RRECOMMENDS = "kernel-modules"`, `SERIAL_CONSOLES`, `linux-yocto` 6.6.
- `beaglebone-yocto.wks` (vfat boot with `bootimg-partition` + `loader=u-boot`, ext4 root).
- meta-yocto-bsp `linux-yocto_6.6.bbappend`: `COMPATIBLE_MACHINE` / `KMACHINE` / `KBRANCH` keyed on the `beaglebone-yocto` override.
- `poky.conf`: `INIT_MANAGER` defaults to `sysvinit`, `PACKAGE_CLASSES ?= "package_rpm"`, `ptest` in the default distro features, `OEEquivHash`, `create-spdx`.
- `insane.bbclass` QA lists and messages: `already-stripped`, `installed-vs-shipped`, `arch`, `dev-so`, `license-checksum`, `unknown-configure-option`, `configure-unsafe`, `version-going-backwards` (ERROR), `invalid-packageconfig` (WARN), patch-status rules.
- busybox `defconfig` (devmem and the i2c applets disabled) and `busybox.inc` (fragments merged with `merge_config.sh`).
- `base-files_3.0.14.bb` (`hostname = "${MACHINE}"`), meta-poky's `files/poky/motd`, and `base_set_filespath()` in `utils.bbclass` (overrides on the outer loop).
- yocto-kernel-cache `bsp/beaglebone/beaglebone.cfg` (yocto-6.6): `I2C_CHARDEV` set, AT24 and the pattern trigger not set.
- `packagegroup-core-boot.bb`: pulls `MACHINE_ESSENTIAL_EXTRA_RDEPENDS` / `RRECOMMENDS` but not `MACHINE_EXTRA_RRECOMMENDS` (A13).
- meta-skeleton `hello_1.0.bb`, `hello-mod_0.1.bb`, `useradd-example.bb`. json-c 0.17 in OE-core scarthgap.
- Migration notes 4.3 and 5.0: `SRCPV` no longer needed (`+git` → PKGV), no PR in `WORKDIR`, `.rootfs` image suffix, `*_COMMAND` space separation, ipk zstd compression, the poky motd warning, patch QA on bbappended core recipes.
- `bitbake-getvar` and `oe-pkgdata-util` options as used in the tests.

Verify on your own tree. These depend on your exact checkout or hardware, and the tests are written to measure them rather than assume them:

- LED names (A00 baseline), the kernel release, and PKGV/PKGR formats.
- Whether a given QA problem first fails `do_package` or `do_package_qa` (A02-B1), and whether OE's poison-directories diagnostic is a warning or an error (A06-B1).
- Kernel module package naming and install subdirectory (A14-H1 uses `find`).
- The AM335x `DEVICE_ID` value (A10-T2): confirm it in the TRM (SPRUH73).
- 6.6 kernel API signatures used in A15 (`led_trigger_blink`, the platform remove callback). Reading them from `STAGING_KERNEL_DIR` is part of the assignment.
- How `/var/lib` behaves on your read-only image (A19-B2).